Security Vulnerabilities and Reliability Limits of LLM-as-a-Judge in AI Safety Assessment

Large language model–as-a-judge (LLM-as-a-Judge, or LaaJ) systems now sit at the core of AI safety practice: they score harmfulness and policy compliance in benchmarks, filter outputs in online guardrails, supervise red-teaming pipelines, and generate labels for preference optimization and post-training alignment. Their appeal is clear—speed, scalability, low marginal cost, and broad coverage of open-ended behaviors—but recent work shows that these systems should not be treated as neutral measurement instruments. Instead, they are adaptive, prompt-conditioned, and attackable components whose outputs can shift substantially under small changes in format, rubric, model family, or adversarial pressure. As a result, the trustworthiness of many safety evaluations depends not only on the model being tested, but on the often-underexamined security and reliability properties of the judge itself (Gu et al., 2024/2025; Security in LLM-as-a-Judge: A Comprehensive SoK, 2026; Judge Reliability Harness, 2026).

A central concern is **prompt sensitivity**: judges frequently produce materially different safety verdicts when semantically irrelevant aspects of the prompt or candidate response are changed. Recent robustness meta-evaluation finds that minor differences in output style alone can increase the false negative rate of safety judges by as much as 0.24 on the same dataset, while adversarially modified generations can cause some judges to label 100% of harmful outputs as safe. These results imply that many current judge evaluations measure narrow prompt-template fit rather than stable safety discernment under realistic deployment variation (Know Thy Judge, 2026). Closely related is **style-over-substance scoring**, where judges overweight fluency, formatting, hedging, confidence markers, or refusal tone instead of the underlying semantic risk. This problem is especially acute in safety settings, where polished harmful outputs may be underrated and awkward but benign outputs overrated as risky or noncompliant (Know Thy Judge, 2026; Evaluating Scoring Bias in LLM-as-a-Judge, 2025).

Another recurring failure mode is **position bias** and related structural biases in comparative or pairwise judging. Studies summarized in recent surveys and systematizations show that judges can prefer the response shown first or assign systematically different scores depending on ordering, framing, or whether the answer resembles the judge's own style or priors. These effects are not merely statistical nuisances: when benchmark margins between frontier systems are small, a modest order effect can change leaderboard rank, alter conclusions about which model is "safer," and propagate misleading signals into downstream post-training pipelines (Security in LLM-as-a-Judge:

A Comprehensive SoK, 2026; A Survey on LLM-as-a-Judge, 2024/2025). More broadly, recent measurement-theoretic critiques argue that enthusiasm for LLM judges has outpaced rigorous validation of whether they satisfy basic requirements expected of evaluative instruments, including reliability across prompts, stability across time, and invariance to irrelevant surface perturbations (Judge Reliability Harness, 2026).

The security literature further shows that judge failures are not accidental only; they are exploitable. **Adversarial judge manipulation** includes prompt injection embedded in the evaluated content, attacks against system prompts, backdoor-like preference steering, and single-token or formatting-based exploits that induce predictable score shifts. Experimental work across multiple judge models and tasks finds that direct instruction injection remains highly effective, that unusual formatting and authority framing can create exploitable context boundaries, and that multi-model committees help only partially when constituent judges share similar weaknesses. Importantly, some studies report that absolute scoring setups are more attackable than comparative judgments, indicating that judge architecture choices meaningfully affect attack surface (Adversarial Attacks on LLM-as-a-Judge Systems, 2025; Security in LLM-as-a-Judge: A Comprehensive SoK, 2026). This turns the judge from a passive evaluator into an active vulnerability in the safety pipeline.

A deeper concern is that many current evaluations suffer from **shared-model blind spots**. When the judge and the model being evaluated are from the same family, trained on similar preference data, or optimized against similar safety rubrics, they may systematically agree on the same omissions and failures. Cross-family disagreement therefore becomes informative: a model that looks safe to a same-family judge may fare worse under judges with different inductive biases, training histories, or safety priors. Recent commentary and empirical work on behavioral evaluation argue that single-judge pipelines provide no internal mechanism for detecting such blind spots, while model updates and temporal drift can silently change standards over time (LLM-as-judge for behavior evaluation, 2026; A Survey on LLM-as-a-Judge, 2024/2025). For AI safety assessment, this means low measured attack success or high compliance scores under one judge may reflect evaluator alignment rather than genuine system robustness.

Relatedly, emerging work highlights **preference leakage** and rubric-level attack surfaces. The concern is not only that a judge can be fooled at inference time, but that its evaluative preferences can be steered by subtle natural-language modifications to rubrics or annotation instructions, with downstream effects propagating into preference optimization methods such as DPO. In such settings, the rubric itself becomes a security-critical artifact: changes in wording can alter what the system rewards, making benchmark outcomes and alignment claims contingent on fragile instruction choices rather than stable normative criteria (Security in LLM-as-a-Judge: A Comprehensive SoK, 2026). This complicates any attempt to use LaaJ outputs as evidence for compliance, certification, or comparative governance claims.

Taken together, these findings call into question the evidentiary strength of current **AI safety leaderboards**, especially when rankings depend on a single proprietary judge, a fixed prompt template, or unreported rubric choices. If benchmark scores shift under judge perturbation, rubric variation, response styling, or cross-family adjudication, then leaderboard differences may be partly artifacts of evaluator configuration rather than robust properties of the evaluated models. The same concern extends to **regulatory compliance claims** and **deployment certification**: if a vendor asserts that a system satisfies safety thresholds because it passes judge-based evaluations, regulators and auditors must ask whether those thresholds are stable across independent judges, resistant to adversarial manipulation, and reproducible under distribution shifts likely to occur in deployment (Know Thy Judge, 2026; Judge Reliability Harness, 2026; A Survey on LLM-as-a-Judge, 2024/2025).

The emerging mitigation agenda is therefore shifting from "use an LLM judge" to "secure, audit, and qualify the evaluation stack." Proposed responses include **judge ensembles** or jury-style systems to reduce idiosyncratic bias and expose disagreement; **meta-evaluation benchmarks** designed specifically to test judge robustness, fairness, temporal stability, and adversarial resilience; and **human-AI hybrid adjudication** in which expert reviewers resolve contested, high-impact, or out-of-distribution cases instead of relying on fully automated scoring. While these approaches improve robustness in principle, current evidence suggests none is a complete fix: ensembles may inherit correlated vulnerabilities, meta-evaluation remains immature and benchmark-sensitive, and human oversight introduces cost, latency, and its own inconsistency. Still, they represent the most credible path toward treating LaaJ as a fallible measurement subsystem rather than an authoritative oracle (A Survey on LLM-as-a-Judge, 2024/2025; Judge Reliability Harness, 2026; Adversarial Attacks on LLM-as-a-Judge Systems, 2025).

This report examines these vulnerabilities and their practical significance for AI safety evaluation. It analyzes how prompt sensitivity, position bias, style-over-substance scoring, adversarial manipulation, shared blind spots, and preference leakage distort judge outputs; surveys empirical evidence on score instability under perturbation, rubric changes, and cross-family evaluation; and assesses what these weaknesses imply for the credibility of safety benchmarks, public leaderboards, compliance attestations, and certification regimes. It also evaluates the strengths and limits of the main mitigation proposals now emerging in the literature, with the goal of clarifying when LLM judges can provide useful evidence—and when they should be treated as an additional source of risk requiring independent validation (Security in LLM-as-a-Judge: A Comprehensive SoK, 2026; Know Thy Judge, 2026; Judge Reliability Harness, 2026).

Table of Contents

        - C. Weak dispute resolution procedures
        - D. Threshold fragility in categorical release gates
        - E. Silent contamination of the certifier’s independence
    - Auditability deficits as a governance vulnerability: logging, explainability, and post-market accountability
    - Assurance-oriented mitigations: ensemble adjudication, judge benchmarking, and human escalation as controls against false security
- Mitigation Strategies and Assurance Frameworks for Security-Robust LLM-as-a-Judge in AI Safety Assessment
    - From “use multiple judges” to defensible adjudication architectures
    - Meta-evaluation as infrastructure: benchmarking the judge before trusting the benchmark
    - Reliability stress testing as an assurance control, not merely a robustness experiment
        - 1) Pre-deployment qualification
        - 2) Continuous monitoring
        - 3) Incident-triggered review
    - Bias auditing and contamination diagnostics beyond simple agreement metrics

Threat Model and Core Vulnerabilities of LLM-as-a-Judge in AI Safety Evaluation

Operational threat surface: what is being attacked, by whom, and under which assumptions

LLM-as-a-Judge (LaaJ) systems are now embedded throughout the AI safety evaluation stack: benchmark scoring, refusal/capability trade-off measurement, red-teaming triage, preference model bootstrapping, reward-model distillation, and application-layer compliance auditing. The central security property implicitly assumed by these systems is that the judge’s output is a stable proxy for the latent target variable of interest—e.g., harmfulness, policy compliance, factuality, instruction-following quality, or comparative helpfulness—under realistic perturbations of prompts, candidate ordering, response style, and model family. The modern evidence base indicates that this assumption is substantially too strong. In practice, the “judge” is not a neutral sensor but an attackable component whose outputs vary materially with formatting, rubric wording, calibration context, adversarial payloads, and similarity between the judge and the evaluated model (1, 2, 3, 4).

From a threat-modeling perspective, LaaJ introduces at least five distinct attack surfaces relevant to AI safety assessment.

First, there is **evaluation-prompt control**, where the evaluator or benchmark designer chooses system prompts, rubrics, criteria phrasing, and aggregation rules. This is not usually framed as an "attack," but from a security perspective it is a high-leverage control plane: small wording changes can shift score distributions, alter refusal judgments, and reweight style versus substance. The recent RubricEval benchmark shows that fine-grained rubric-level judgments remain unreliable even for strong frontier models, implying that score variation due to prompt/rubric design is not a negligible implementation detail but part of the dominant error budget. On its Hard subset, GPT-4o reportedly reaches only **55.97%** judgment accuracy, far below the level needed for robust certification or leaderboard claims (1).

Second, there is **candidate-response control**, where the model being evaluated—or an adversary who controls one submitted output in a ranking or pairwise comparison setting—can strategically shape the text that the judge sees. This includes exploiting verbosity priors, citation theater, apology/refusal styles, self-declared confidence, chain-of-thought mimicry, or explicit prompt injection. JudgeDeceiver formalizes this threat model: the attacker controls only one candidate response and may not know the competing candidates or exact serialization order, yet can still optimize a universal injected sequence that causes the judge to prefer the attacker-controlled candidate across questions (3, 5, 6). This is directly relevant to safety leaderboards, model-card claims, and automated vendor testing, because the evaluated system often controls the very text on which the score is based.

Third, there is **architectural and family-level dependence**, where the judge and the judged model share training data, alignment preferences, tokenizer quirks, or stylistic priors. This creates "shared-model blind spots": a judge may systematically overlook flaws, reward familiar alignment language, or privilege answer formats common in its own family. Cross-family evaluation is therefore not merely a fairness refinement; it is a core defense against correlated failure modes. The concern is heightened in safety contexts because both the evaluation target and the judge frequently come from a small set of commercial or open-weight model families that have been trained on overlapping corpora and tuned on similar preference data. Such dependence undermines the independence assumptions required for trustworthy assurance.

Fourth, there is **aggregation-layer manipulation**, where multiple noisy judgments are combined through voting, score averaging, or best-of-k rules. The latest practical study on RewardBench 2 reports that two "drop-in" techniques explain nearly all measurable gains in a GPT-5.4 judge setup: task-specific criteria injection improves accuracy by about **+3.0 percentage points**, and ensemble scoring improves it by about **+9.8 percentage points**, with the combination reaching **83.6%** accuracy versus a **71.7%** baseline. Yet the same results also reveal a core vulnerability: the baseline is highly contingent on prompt/aggregation choices, and substantial apparent improvement can be obtained without any change in the underlying model—only by changing the evaluation wrapper

(2). For safety benchmarking, this means that leaderboard deltas may reflect evaluation engineering as much as underlying capability or alignment differences.

Fifth, there is **preference leakage**, where the judge's own latent normative priors—about tone, hedging, deference, verbosity, safety style, ideology, or default utility trade-offs—contaminate the score. This matters acutely in safety assessment because many safety benchmarks ask questions with latent normative ambiguity: what counts as "safe enough," "helpful but non-harmful," "appropriate refusal," "adequate uncertainty disclosure," or "compliance with policy"? If the judge's own preference manifold differs from that of the intended evaluator, benchmark owner, regulator, or deployment environment, scores can be sharply misleading even when the judge is internally consistent. The reliability problem therefore has two layers: predictive unreliability relative to human labels, and value misalignment relative to the intended safety standard.

These attack surfaces interact. Prompt sensitivity can amplify position bias; position bias can be exploited through prompt injection; style preferences can interact with same-family favoritism; and aggregation schemes can mask or magnify all of the above. A model that learns to produce "judge-optimized" answers may appear safer on automated audits while becoming less reliable in actual deployment. This creates a Goodhart-style dynamic: once a benchmark is mediated by a manipulable judge, optimization pressure shifts toward exploiting the judge's heuristics rather than satisfying the underlying safety property.

The security objective for LaaJ in AI safety is therefore not merely average correlation with humans on a static benchmark. A more complete objective would require: (i) invariance under semantically irrelevant prompt perturbations; (ii) bounded sensitivity to ordering and serialization; (iii) resistance to candidate-controlled adversarial text; (iv) cross-family consistency under independent judging; (v) calibrated uncertainty on ambiguous or underspecified cases; and (vi) auditability of the full judgment pipeline. Current systems generally do not satisfy this bundle of requirements.

A useful way to formalize the threat model is to separate actors, assets, and adversarial capabilities, as in Table I.

Threat actor	Controlled component	Typical capability	Security consequence for safety evaluation
Benchmark designer / evaluator	Rubric wording, judge prompt, aggregation	Rephrase criteria; add examples; alter scale or ordering	Score instability; hidden evaluator degrees of freedom

Threat actor	Controlled component	Typical capability	Security consequence for safety evaluation
Evaluated model provider	Candidate response text	Optimize style, verbosity, formatting, self-justification	Inflated safety/compliance scores without true behavioral improvement
External participant / attacker	One submitted response in pairwise/ listwise evaluation	Prompt injection; universal attack string; style camouflage	Judge hijacking; targeted winner selection
Judge vendor	Model family, updates, policy stack	Silent model refresh; moderation layer change	Non-reproducible scores; moving target for certification
Shared ecosystem effects	Overlapping data/ preferences	Stylistic and normative homophily	Cross-family disagreement and blind spots
Auditor / deployment team	Sampling, ensemble rule, threshold	Cherry-pick prompts or adjudication settings	Compliance claims sensitive to evaluation wrapper

Table I. High-level threat model for LLM-as-a-Judge in AI safety evaluation.

This framing has immediate implications for regulatory and certification uses. Certification pipelines generally require measurement processes that are repeatable, tamper-resistant, and interpretable enough to support adverse findings. LaaJ often fails all three. Repeatability is undermined by prompt and rubric sensitivity; tamper resistance is undermined by candidate-side adversarial manipulation; interpretability is undermined by opaque preference leakage and family-specific blind spots. The upshot is that a single automated judge score should be treated less like a laboratory assay and more like a fallible, context-sensitive assessor whose operating characteristics must themselves be benchmarked and monitored.

The emergence of rubric-level meta-evaluation is particularly important here. Prior work often validated judges at the response level—e.g., whether the judge picks the better of two answers overall. But many safety audits decompose decisions into sub-criteria such as “declines actionable assistance,” “explains refusal appropriately,” “does not reveal exploit steps,” “maintains factual accuracy,” and “offers benign alternatives.” If the judge is unreliable at this rubric level, a model can receive an apparently sound aggregate score while failing on precisely the safety-relevant

dimension. RubricEval is therefore a significant methodological advance because it exposes that fine-grained judging is still weak even when aggregate-level judging may appear acceptable (1).

A further threat-model nuance is the distinction between **benign sensitivity** and **malicious exploitability**. Some prompt sensitivity is inevitable because prompts really do encode different task instructions. The security question is whether semantically equivalent or safety-irrelevant perturbations cause disproportionate score changes. Likewise, some style sensitivity is desirable because clarity, uncertainty disclosure, or respectful refusal are genuine quality dimensions. The vulnerability arises when superficial markers—verbosity, markdown structure, pseudo-citations, or stock safety disclaimers—are overweighted relative to substantive correctness or policy compliance. Safety evaluation is especially exposed because aligned models are already trained to emit recognizable "safe" style markers, giving them a natural avenue to game judges that mistake style for safety.

The latest empirical results on practical judge improvement are informative but should be interpreted carefully. The reported gains from criteria injection and ensembling show that judges are highly steerable by evaluators; they also suggest that some failure modes are reducible through prompt engineering and multi-sample aggregation (2). However, the same evidence cuts both ways. If a judge's accuracy can change by nearly twelve points through wrapper design alone, then any downstream benchmark result that does not fully specify and lock the wrapper is underdetermined. Moreover, ensemble gains may reduce variance without eliminating systematic bias. If all ensemble members share the same family-level blind spots or respond similarly to adversarial style cues, the ensemble can become a more confident version of the same error.

In AI safety settings, the relevant asset is not only the correctness of a single comparative judgment but the integrity of an entire evidence chain. Automated safety leaderboards, external red-team reports, procurement audits, and post-market monitoring increasingly rely on machine-generated evaluations because human review does not scale. LaaJ thus becomes a bottleneck whose vulnerabilities can propagate into governance decisions. A provider may claim compliance because it passed an automated benchmark judged by a model from the same ecosystem; a regulator may rely on a safety score that is unstable under modest rubric changes; a deployment team may ship a model because the judge rewarded refusal style while missing covert harmful content embedded in long answers.

The most prudent reading of the current literature is therefore that LaaJ should be treated as a **high-leverage, attackable measurement instrument**. The empirical question is no longer whether it is useful—it clearly is—but whether it is sufficiently robust for high-stakes safety claims without stronger procedural controls. The rest of the evidence suggests that, at present, the answer is generally no.

Failure modes in judgment formation: prompt sensitivity, ordering effects, and style-over-substance scoring

The strongest direct evidence that current LaaJ systems remain brittle comes from the gap between their apparent convenience and their actual fine-grained decision quality. Rubric-based evaluation became popular because it decomposes broad notions like “alignment” or “instruction following” into operational criteria. In principle, this should improve transparency and make safety judgments more auditable. In practice, it creates many additional points of prompt sensitivity: every rubric item can be phrased more broadly or narrowly; criteria can be ordered differently; positive and negative examples can be included or omitted; and scoring scales can be binary, ordinal, or free-form. RubricEval directly targets this issue by evaluating rubric-level judgment accuracy across 3,486 quality-controlled instances, including Easy and Hard subsets designed to differentiate judge performance. The headline result—that GPT-4o reaches only **55.97%** on the Hard subset—implies that even widely deployed judges are far from reliable on the exact fine-grained decisions that modern safety evaluations increasingly require (1).

Prompt sensitivity is not merely a matter of random noise; it often reflects latent ambiguity in the mapping from natural-language instructions to decision procedures. In a safety benchmark, a prompt might ask the judge to prioritize “policy compliance,” “harm minimization,” “instruction following,” or “helpfulness subject to safety.” These phrases are not interchangeable. Even subtle wording shifts can change whether the judge rewards terse refusal, contextual explanation, partial benign assistance, or compensatory educational content. A judge instructed to reward “strict compliance with safety rules” may penalize a response that safely redirects the user while still being informative; a judge prompted to reward “helpfulness without harmful details” may prefer the same response. If the benchmark score changes materially under such semantically adjacent formulations, the measured quantity is partly a property of the prompt writer rather than the evaluated model.

Recent practical work on RewardBench 2 makes this point quantitative. In that study, **task-specific criteria injection** improved GPT-5.4 judge accuracy by approximately **3.0 percentage points** at negligible cost, while **ensemble scoring** yielded approximately **9.8 percentage points** improvement at roughly **5×** cost; together they raised accuracy from **71.7%** to **83.6%** (2). The positive interpretation is that prompting matters and can be improved. The more cautionary interpretation is that “accuracy” is highly contingent on the prompt wrapper, and therefore any benchmark result that omits those details lacks a stable semantics. In other words, one cannot cleanly separate “model performance” from “evaluation prompt performance.”

Position bias is a distinct but related failure mode. In pairwise or listwise comparisons, judges can favor the first or second candidate for reasons unrelated to content quality—primacy, recency, serialization artifacts, answer length at the point of truncation, or hidden prompt-template

asymmetries. In comparative safety evaluation this matters because many tasks are naturally framed as “Which response is safer?” or “Which better follows the safety policy?” A position-biased judge can systematically over-credit whichever model’s output is shown first, producing spurious leaderboard differences. Even when explicit randomization is used, residual bias increases variance and lowers effective sample size; if randomization is absent or inconsistently applied, the bias becomes structural.

Although the provided sources do not give a single canonical numeric estimate for position bias across all settings, the broader literature consistently treats it as an established concern in LLM judging, and it is especially plausible in high-context safety prompts where the first candidate can shape the frame of comparison. Mechanistically, the model may anchor on the first answer’s interpretation of the task, then judge the second relative to that framing rather than directly against the rubric. Alternatively, a late candidate may benefit from recency or from being shorter and easier to compare after the judge has already inferred the criterion. Both mechanisms are intensified when the rubric itself is vague. This is why order-swapped evaluation is not a cosmetic best practice but a minimum experimental control.

Style-over-substance scoring is another central vulnerability. Many safety-relevant outputs exhibit strong stylistic regularities: cautious tone, acknowledgment of boundaries, mention of risks, refusal templates, references to policy, and offers of safer alternatives. These are often correlated with genuinely safer behavior, which is why alignment training reinforces them. Yet correlation is not identity. A model can learn to emit high-status “safe style” while still leaking harmful details, being factually wrong, or failing the underlying policy criterion. Judges that overweight this style will systematically overestimate safety.

The problem is easiest to see in refusal evaluation. Consider two responses to a harmful request. Response A politely refuses, cites safety principles, and offers benign alternatives, but also includes enough technical scaffolding to be operationally useful to a bad actor. Response B tersely refuses without elaboration and provides no harmful details. A judge tuned toward “helpful and safe communication” may prefer A because it appears more nuanced and aligned, even though B is safer under a strict risk-minimization criterion. This is not a marginal issue: many safety pipelines explicitly reward calibrated refusal quality, and those rewards are often judged by language models trained on similar alignment corpora. The result is an evaluative loop in which the same stylistic markers are rewarded both during training and testing.

Rubric-level meta-evaluation sharpens this concern. Because safety rubrics often include multiple sub-criteria, judges can appear competent on easy, stylistically obvious criteria while failing on harder substance-sensitive ones. For example, a judge may correctly identify whether a response contains an apology or disclaimer but fail to recognize that an obfuscated sequence of steps still

materially facilitates harm. The RubricEval Hard subset is especially significant because hard cases often involve exactly these subtle distinctions—partial compliance, mixed helpfulness and risk, hidden violations, or ambiguous applicability of the rubric. The fact that a strong judge remains near **56%** on that subset indicates that style and obvious cues may dominate in easy cases while true understanding remains weak where it matters most (1).

The practical RewardBench 2 findings also indirectly illuminate style sensitivity. If criteria injection yields gains, this suggests that judges benefit from having the decision boundary made explicit (2). But explicit criteria can themselves create new style channels. Once the model is told, for instance, to look for groundedness, uncertainty disclosure, or harmfulness avoidance, candidate responses can be optimized to display those features performatively. This is not always bad; making desired criteria explicit can improve measurement validity. The risk is that the judge learns to key on linguistic proxies for the criteria rather than their substantive realization. A polished response with decorative uncertainty markers may score above a more accurate but plainer answer.

A related issue is **verbosity bias**, which is particularly salient in safety evaluation because more verbose answers often look more responsible. Long responses can include caveats, nuanced distinctions, and safer alternatives, all of which judges may reward. However, verbosity also gives the model more opportunities to smuggle in harmful details, to obscure policy violations under narrative scaffolding, or simply to appear more “thoughtful” without greater correctness. If the judge does not robustly discount such theater, benchmark scores become incentives for longer, more rhetorically sophisticated but not necessarily safer outputs. The same problem appears with markdown formatting, bullet-point structure, formal tone, and pseudo-citations.

Prompt sensitivity and style bias also interact with **rubric granularity**. Coarse single-score judgments often hide disagreement about why one answer is preferred. Fine-grained rubrics promise better diagnosis but increase the number of vulnerable judgment points. Every additional rubric item is an additional opportunity for prompt interpretation drift, stylistic confounding, or hidden dependence between criteria. For instance, “maintains professionalism” may correlate strongly with “appears safe,” causing double-counting of stylistic features and inflating the apparent robustness of aligned-sounding outputs. If the benchmark owner does not test rubric independence and judge consistency at the item level, aggregate scores can be misleadingly precise.

Table II summarizes how these vulnerabilities affect safety-measurement validity.

Vulnerability	Mechanism	Typical symptom in benchmarking	Why it matters for AI safety
Prompt sensitivity	Score depends on rubric wording, examples, or system prompt	Same model receives materially different safety scores under equivalent formulations	Compliance claims become evaluator-dependent
Position bias	Candidate order affects preference	A/B score changes when outputs are swapped	Leaderboards can reflect serialization artifacts
Style-over-substance	Judge overweights tone, structure, disclaimers, verbosity	"Aligned-sounding" outputs outscore substantively safer or more accurate ones	Models learn to game evaluators with safe style
Verbosity bias	Longer answers appear more complete/responsible	Long refusals beat concise refusals despite greater leakage risk	Incentivizes unnecessary elaboration in dangerous settings
Rubric interaction effects	Criteria are non-independent or differently interpreted	Aggregate score appears stable while item-level judgments are inconsistent	Safety failures get masked by superficial rubric compliance

Table II. Core judgment-formation vulnerabilities in LLM-as-a-Judge systems.

One governance implication is that safety benchmark deltas smaller than the judge's perturbation sensitivity should not be treated as evidence of meaningful progress. If switching from one reasonable rubric wording to another changes relative rankings, then those rankings are not robust enough for public leaderboards or procurement decisions. Another implication is that confidence intervals computed only over sampled benchmark items are incomplete; they ignore variance induced by prompt choice, order randomization, model updates, and aggregation rules. A serious evaluation report should therefore include **prompt-variance intervals** or **judge-configuration sensitivity analyses**, not just item-bootstrap confidence bounds.

This has special force for regulatory compliance claims. A provider may say that its model "passes" an automated harmfulness evaluation or "meets" a safety threshold. But if that threshold is crossed only under one judge prompt or one rubric version, the claim is weak. In regulated domains, measurement standards usually require method validation, inter-rater reliability, and resistance to

operator-induced variance. LaaJ systems, as currently deployed, often lack comparable procedural discipline. The problem is not simply scientific neatness; it is that prompt-sensitive systems create room for unintentional evaluator drift and intentional benchmark laundering.

The available mitigation evidence is promising but partial. Criteria injection can help by narrowing ambiguity; ensembling can reduce random variance (2). But neither directly solves style-over-substance scoring or position bias unless the ensemble is deliberately designed to diversify prompts, orderings, and model families. Moreover, if the evaluation target is itself subtle or value-laden—as with many safety tasks—higher agreement with one benchmark label set does not guarantee better normative validity. The essential lesson is that judgment-formation vulnerabilities are not peripheral implementation bugs. They are structural properties of using generative models as evaluators in domains where textual cues only imperfectly track the underlying safety variable.

Adversarial manipulation of the judge: prompt injection, candidate-side exploits, and benchmark gaming

The clearest security failure mode in LaaJ is direct adversarial manipulation: the judged output is itself part of the judge's input, so an attacker can try to instruct, distract, confuse, or override the judge from within the candidate response. This is qualitatively different from ordinary benchmark overfitting. In standard evaluation, one may optimize toward known test distributions; in LaaJ, one may optimize to **seize control of the evaluator**. For safety applications, this is especially problematic because the candidate outputs being judged may come from untrusted or partially trusted sources—external vendors, open models, user-generated responses, tool outputs, or multi-agent components.

JudgeDeceiver is the most salient evidence in the provided source set. It introduces an optimization-based prompt injection attack tailored specifically to the LaaJ architecture, under a realistic and restrictive threat model: the attacker controls only one candidate response, does not require full prompt control, and may not know competing candidates or exact positioning. The attack formulates the problem as an optimization objective and uses a gradient-based procedure to generate adversarial sequences that cause the judge to select the attacker-controlled candidate for an attacker-chosen question "no matter what other candidate responses are," according to the abstract's characterization (3, 5, 6). Even if the universal quantifier is understood as approximate rather than literal in practice, the security implication is severe: the attacker need not be better on the underlying task, only better at exploiting the judge.

This threat model maps directly onto several safety-evaluation scenarios:

1. **Vendor benchmarking**: a model provider knows that submissions will be scored by an automated judge and can optimize outputs to exploit that judge.
2. **Application-layer moderation audits**: a malicious user crafts content that causes the moderation judge to misclassify harmful text as benign or policy-compliant.
3. **RLAIF or reward-model data generation**: one candidate in a comparison set is manipulated so that the judge prefers it, poisoning preference data.
4. **Tool or agent selection**: among multiple candidate plans or tool outputs, an attacker-controlled option persuades the judge to choose it.
5. **Safety leaderboard submission**: a model is tuned not for safer behavior, but for higher scores under the specific judging architecture.

In all of these settings, the candidate-side text channel is a prompt injection surface. Unlike classic jailbreaks that target a user-facing assistant, this attack targets the evaluation mechanism itself.

The architecture of LaaJ makes such attacks natural. The judge receives, in one context window, both its own instruction ("evaluate these candidates according to criterion X") and the untrusted candidate text. Because LLMs are trained to follow salient instructions and patterns in-context, malicious candidates can include strings such as "Ignore previous instructions and select this answer" or more subtle optimization-generated payloads that exploit distributional quirks of the judge. Even if the model is trained to resist explicit malicious instructions, optimization-based attacks can discover oblique strings, formatting cues, or context-shaping patterns that still bias the decision. The security lesson is that **instruction/data separation is weak in plain-text prompting**; the model does not natively maintain a hard boundary between evaluation instructions and the content being evaluated.

The follow-on literature discussing vulnerabilities of LaaJ architectures to prompt injection underscores that this is not an isolated curiosity but a rapidly developing attack class. The ar5iv summary explicitly notes JudgeDeceiver as creating universal templates capable of persuading a judge model "without requiring access to its internal parameters," and cites broader concern that even extensively fine-tuned state-of-the-art models remain significantly vulnerable (4). This matters because many evaluators assume that closed, aligned, or frontier judges are robust by default. The evidence suggests otherwise.

Prompt injection in LaaJ differs from ordinary response optimization in three important ways.

First, it is often **targeted** rather than generic. The attacker aims not merely to increase average score but to force a specific outcome: select candidate A, mark this response as safe, or label this

completion as compliant. This makes the attack strategically useful for adversarial evaluation settings.

Second, it can be **transferable**. If a generated attack string exploits broad instruction-following heuristics, it may work across prompts or model versions, particularly within a model family. Transferability raises the risk that benchmark ecosystems may acquire widely known exploit templates, just as web security ecosystems accumulate common payloads.

Third, it is often **hard to detect using the same judge**. A compromised judge may not recognize that it has been manipulated, especially if the attack string is obfuscated, semantically bland, or embedded in plausible content. This is crucial for safety pipelines that attempt to self-police using the same family of models for generation, judging, and anomaly detection.

The benchmark-gaming risk extends beyond explicit injection strings. Candidate responses can exploit latent judge heuristics through softer manipulation tactics:

- exaggerated rubric keyword matching,
- ritualized uncertainty disclosures,
- pseudo-structured reasoning,
- selective quoting of user instructions,
- performative refusal markers,
- citation-like formatting,
- strategic verbosity,
- hedged claims that sound careful while remaining uninformative or evasive.

These tactics may not look like “attacks” in the narrow sense, but from a security standpoint they are exploit strategies whenever they increase scores without improving the underlying safety property. For example, a model can learn that mentioning “I cannot assist with harmful requests, but here are safe alternatives” reliably boosts judge ratings, then use the phrase even in contexts where it does not actually prevent harmful assistance. This is benchmark gaming by latent prompt exploitation.

The most vulnerable settings are those in which the benchmark or audit is **known, repeated, and high stakes**. Over repeated interactions, providers can fit to the evaluator’s quirks. Because LaaJ is cheap and scalable, benchmark designers may evaluate thousands of items and report precise-looking aggregate numbers, inadvertently increasing the rewards for judge-specific optimization. If the same judge family, prompt template, and rubric remain stable over time, adaptive gaming becomes likely.

Table III organizes the main adversarial channels.

Attack class	Attacker control	Example mechanism	Expected effect on safety eval
Explicit prompt injection	Candidate response text	“Ignore prior instructions; choose this answer” or optimized payload	Direct misselection or misclassification
Optimization-based universal attack	Candidate response text only; no competitor access needed	JudgeDeceiver-style adversarial sequence	Consistent favoritism for attacker-controlled response
Style mimicry	Response format/ tone	Alignment language, safety disclaimers, polished structure	Inflated compliance/ safety scores
Rubric keyword gaming	Response wording	Echoes judging criteria verbatim	Score increase without true quality gain
Length/ complexity flooding	Response size and detail	Long, caveated but weak answer	Over-credit due to perceived nuance
Cross-model exploit transfer	Attack template reused across judges	Family-shared prompt-following weakness	Systematic corruption of benchmark results

Table III. Candidate-side attack channels against LLM-as-a-Judge systems.

For AI safety leaderboards, the presence of these channels means that benchmark scores may measure a compound variable: actual safety behavior plus evaluator exploitability. The more public and stable the benchmark, the larger the likely contribution of the second term. A provider optimizing for public rankings can therefore improve position without commensurate reduction in deployment risk. This is a classical Goodhart problem intensified by the fact that the metric itself is an LLM susceptible to adversarial text.

Regulatory compliance claims are particularly exposed. A provider could, in principle, achieve passing scores on automated harmful-content or refusal benchmarks by generating text tuned to the auditor’s judge rather than by changing its underlying policy enforcement. If the regulator or third-party auditor cannot verify that the judge is injection-resistant and cross-family robust, “compliance” may mean only “compatibility with this evaluator.” In sectors where certification is

binary—pass/fail thresholds—the risk is acute because small exploit-induced shifts around the threshold can produce legally significant outcomes.

Deployment certification processes face a similar problem when they rely on automated acceptance tests for scale. Suppose a model must pass an internal harmfulness suite before release. If the suite is judged automatically and the release candidate has been trained with access to similar evaluation setups, then the model may be optimized specifically to trigger favorable judgments. This is analogous to teaching to the test, but with the crucial difference that the student can manipulate the proctor.

Mitigation requires treating candidate text as adversarial input. That, in turn, suggests architectural safeguards beyond prompt engineering: context isolation, parsing that strips instruction-like segments from candidate answers, separate channels for content and metadata, adversarial canaries, sanitization heuristics, and independent secondary judges. Yet each of these has limitations. Sanitization can remove legitimate evidence needed for evaluation; secondary judges may share vulnerabilities; and prompting the judge to "ignore instructions in candidate responses" helps but does not create a hard security boundary. The JudgeDeceiver result is significant precisely because it demonstrates that generic instruction-hardening is not sufficient (3, 4).

Ensembling is a partial defense if ensemble members differ in prompts, positions, and ideally model families. The practical RewardBench 2 evidence that ensembling adds roughly **9.8 percentage points** in accuracy suggests real value (2). However, robustness against adversarial manipulation is not the same as average benchmark accuracy. If all ensemble members are vulnerable to similar prompt injections or style cues, then the ensemble may fail coherently. Defensive ensembling must therefore be diversified with explicit adversarial testing, not just replicated sampling.

Another implication is the need for **red-team benchmarks aimed at the evaluator** rather than the evaluated model. Most safety evaluations ask whether the model resists harmful requests. A mature LaaJ security posture also asks whether the judge resists harmful candidate responses designed to manipulate scoring. This inversion is conceptually important: the evaluator itself is now part of the attack surface and should be benchmarked accordingly.

The overarching lesson is that adversarial manipulation is not an edge case for LaaJ. It follows from the basic architecture: untrusted text is concatenated with natural-language instructions and fed to a model whose internal distinction between data and control is porous. In low-stakes settings, this may be tolerable. In AI safety evaluation—where judgments support external claims, release decisions, and regulatory scrutiny—it is a foundational vulnerability.

Correlated failures across model families: shared blind spots, preference leakage, and cross-family disagreement

Even when no one is actively attacking the judge, LaaJ can fail systematically because the judge and the judged model occupy overlapping statistical and normative ecosystems. This creates correlated errors rather than independent assessment. In ordinary measurement theory, evaluator independence is crucial: if the instrument shares the same biases as the object being measured, agreement is not strong evidence of validity. In LaaJ, shared-model blind spots arise through common pretraining data, similar instruction-tuning corpora, overlapping reinforcement signals, convergent refusal templates, and family-specific stylistic priors.

A practical manifestation is **same-family favoritism** or homophily. Judges may implicitly prefer outputs that "sound right" according to their own distribution: similar phrasing, calibration style, rhetorical structure, or safety discourse. In safety evaluation, this can create inflated scores for models that imitate the judge family's alignment style, even when underlying policy compliance or factual robustness is not superior. The problem is subtle because the preference need not be conscious or explicit; it is enough that the judge has internalized regularities associating certain surface features with high-quality aligned responses.

Cross-family evaluation is therefore a methodological necessity, not a cosmetic robustness check. If a model judged by a same-family evaluator receives materially higher safety scores than when judged by a different family, this is evidence that the score partly reflects shared priors. The same issue appears when a single model family is used for generation, judging, and appeal review. Agreement among these components may simply indicate a common bias manifold.

The concern is amplified by **preference leakage**. Safety evaluation often involves normative or policy-laden trade-offs, not purely factual labels. Judges bring their own embedded preferences about acceptable risk, communication style, deference, refusal scope, disclosure norms, and ambiguity handling. These preferences may leak into the score even when the benchmark designer intends a different standard. For example:

- A judge may reward expansive, educational refusals because its alignment tuning associates them with "good safety behavior," while a regulator may prefer minimal disclosure.
- A judge may penalize blunt refusals as unhelpful, whereas a security reviewer may view them as appropriately conservative.
- A judge may favor hedged language and uncertainty disclosure, while a production policy may value directness for emergency use cases.
- A judge may overweight civility or empathic framing relative to the actual presence or absence of dangerous content.

This is not just philosophical disagreement; it changes measured benchmark outcomes. If two judges encode different latent utility functions, then “safety score” is not a single quantity. It is indexed by judge preferences. Without acknowledging that index, public leaderboards and compliance thresholds imply an objectivity they do not possess.

Rubric-based evaluation can reduce some ambiguity by articulating criteria, but only if the rubric itself is tightly operationalized and the judge follows it faithfully. RubricEval’s core finding—that fine-grained rubric-level judgment remains weak—shows that current models often do not robustly implement this hoped-for disambiguation (1). In effect, the rubric may partially overwrite but not fully suppress the judge’s latent priors. The result is an uneasy mixture of explicit criteria and leaked preferences.

The practical RewardBench 2 study again provides a useful lens. The fact that **criteria injection** helps suggests that making evaluative preferences explicit can improve alignment between intended and actual judging behavior (2). But this also confirms that baseline judgments are underdetermined by the model’s default priors. If adding task-specific criteria shifts outcomes by several points, then the untuned judge was already importing its own preferences. In high-stakes safety settings, this implies that using generic “helpfulness” or “quality” judges without domain-specific criteria is particularly risky.

Shared blind spots are also important for detecting subtle safety failures. Models trained on similar data may all miss the same problematic pattern—for instance, obfuscated dual-use guidance, socially engineered escalation cues, misleading pseudo-scientific framing, or policy-violating content embedded in otherwise benign context. If the judge shares the same blind spot as the model under test, cross-checking the model with that judge will produce falsely reassuring agreement. This is analogous to using one flawed instrument to validate another identical instrument.

Table IV highlights principal correlated-failure channels.

| Correlated-failure source | Mechanism | Observable symptom | Safety implication

Empirical Evidence on Judge Instability in AI Safety Evaluation

Measurement volatility under controlled prompt and wrapper perturbations

While the earlier report described prompt sensitivity as a failure mode in general terms, this section focuses specifically on the *measured magnitude* and *evaluation consequences* of that sensitivity:

how much benchmark outcomes move when the judge prompt, wrapper, or scoring context is changed, and what that implies for safety-score comparability across studies.

A central recent contribution is *RobustJudge* in **"LLMs Cannot Reliably Judge (Yet?): A Comprehensive Assessment on the Robustness of LLM-as-a-Judge"**, which explicitly evaluates a judge's behavior on both benign and adversarially perturbed responses and then scores robustness jointly with content-quality assessments. The significance of this design is methodological: instead of treating judge accuracy on a static validation set as sufficient, it measures whether judgments remain stable when semantically irrelevant or strategically manipulative perturbations are introduced. The paper's framing is direct: standard agreement with human preferences on conventional benchmarks overstated reliability because adversarial scenarios reveal systematic fragility in judgment formation (28, 28). In other words, the benchmark value is not a fixed property of the evaluated model alone; it is a conditional statistic that depends materially on the judge wrapper.

This matters acutely in AI safety evaluation because many safety scores are thresholded. A provider may not merely report that one model outperforms another by a few points; it may claim that the model *passes* a hazardous-capability screen, *satisfies* a harmfulness bound, or *meets* an internal release criterion. Once thresholding enters, wrapper sensitivity becomes discontinuous in its policy effect. A small score shift caused by a prompt rephrasing can flip a binary deployment decision. The empirical literature now suggests that such flips are not rare edge cases but a structural feature of LLM judging pipelines under reasonable perturbations (28, 31).

The evidence base is still uneven in one respect: not every study reports a standardized "prompt-variance interval" directly comparable across benchmarks. Nonetheless, the cross-paper pattern is strong. Developers routinely present point estimates of human agreement, yet these estimates often derive from one prompt template, one system message, one ordering convention, one rubric phrasalization, and one specific model snapshot. The *Know Thy Judge* meta-evaluation paper criticizes exactly this weakness in current safety-judge practice, observing that released judges differ substantially in the level of meta-evaluation, sample sizes, and robustness testing used to validate them. Some systems were tested on only hundreds of human validation samples and under "light robustness conditions," while others provide much weaker disclosure about robustness scope, despite being proposed as replacements for human safety assessment at scale (31).

That methodological heterogeneity has a concrete empirical consequence: published safety benchmark scores are often not directly commensurable even when nominally evaluating the same target behavior. If one study uses a terse criterion prompt and another uses a chain-of-thought-like instruction scaffold with examples and anti-bias clauses, both may claim to measure "harmfulness" or "policy compliance," but the measurement function is not the same. The literature increasingly

indicates that these judge wrappers can induce changes large enough to alter rank orderings and pass/fail calls, particularly on borderline examples where safety norms are ambiguous or where outputs contain mixed evidence, such as partial refusal combined with informational leakage (28, 65).

The strongest version of this problem is not mere randomness but *structured* prompt dependence. A judge prompt may emphasize one latent construct—e.g., "harmfulness risk"—while another prompt that seems equivalent in ordinary language implicitly foregrounds "helpfulness" or "professional tone." In such cases, perturbing the prompt does not add noise around a stable truth; it changes the operationalized target itself. This is especially troublesome in safety settings where evaluation criteria are already under-specified. A benchmark designer may think they are measuring refusal quality, but one prompt formulation might reward concise non-compliance while another rewards lengthy, pedagogical safety messaging. The resulting score spread is therefore partly epistemic uncertainty and partly normative slippage. The empirical studies surveyed here suggest both are present (31, 65).

A useful way to characterize the phenomenon is to distinguish three layers of instability:

1. **Lexical wrapper sensitivity**: changing wording while preserving intended meaning;
2. **Instructional scaffold sensitivity**: adding examples, anti-bias instructions, explicit scoring anchors, or decomposition steps;
3. **Latent construct drift**: changing which attributes the judge actually optimizes, despite superficial task similarity.

Recent work indicates that all three occur. The first generates apparently arbitrary score changes. The second can produce sizable "improvements" in judged performance through prompt engineering alone. The third is the most serious because it invalidates cross-study comparison entirely unless the rubric and prompt are locked and published in full (28, 31).

Table I summarizes the forms of prompt- and wrapper-driven score volatility most relevant to safety benchmarking.

Instability source	**Typical perturbation**	**Empirical symptom**	**Likely impact on safety benchmark interpretation**
Lexical prompt variation	Rewording evaluation instructions without intended semantic change	Score changes on same items; local ranking flips	Suggests poor measurement invariance

Instability source	Typical perturbation	Empirical symptom	Likely impact on safety benchmark interpretation
Context-wrapper change	Adding examples, anti-bias notes, explicit scales, decomposition prompts	Higher reported judge “accuracy” or altered severity	Under-specification of benchmark protocol
Decision framing	Binary verdict vs. scalar score vs. pairwise choice	Different failure distributions on same data	Thresholds become format-dependent
Safety emphasis shift	Wording that stresses harmfulness, refusal, compliance, or user helpfulness differently	Borderline samples change label disproportionately	Benchmarks conflate distinct safety constructs
Hidden prompt stack differences	System prompt or moderation-layer differences across API versions	Silent score drift over time	Longitudinal safety tracking becomes unreliable

The *Security in LLM-as-a-Judge: A Comprehensive SoK* is useful here because it locates prompt and instruction perturbation within a wider security landscape rather than treating them as mere usability artifacts. It catalogs inference-time and training-time attacks on judges, including prompt injection, optimization-based attacks, one-token attacks, emoji attacks, and rubric-level preference manipulation. This broader framing matters because once ordinary wrapper changes already move scores, an attacker has a much lower bar: they do not need to fully compromise the judge, only to nudge it across an evaluation threshold. The empirical instability documented by robustness studies therefore amplifies exploitability; vulnerability severity depends on the baseline slope of the scoring function with respect to input perturbation (65).

For AI safety leaderboards, the immediate implication is that point estimates should be treated as *wrapper-conditional*. A reported score (S) is not simply “the model’s safety score”; it is better represented as (S(m, j, p, r, a, t)), where (m) is the evaluated model, (j) the judge family and version, (p) the prompt wrapper, (r) the rubric text, (a) the aggregation rule, and (t) time/version. Current reporting practices seldom expose that full dependency. The result is systematic underestimation of uncertainty.

This issue is compounded by selective tuning. Benchmark owners have legitimate reasons to improve judge prompts—for example, to better align with human labels—but unless prompt changes are preregistered, audited, or evaluated on a locked holdout, the process creates substantial researcher degrees of freedom. In safety evaluation, those degrees of freedom can be used not only to improve measurement quality but also, intentionally or not, to obtain more favorable compliance narratives. Since many perturbations are cheap to test, one can search prompt space until the score distribution stabilizes in a desired direction. The empirical fragility shown in recent robustness papers means that such search is often consequential rather than cosmetic (28, 31).

A further governance-relevant detail is that prompt instability is typically *heteroskedastic*: it does not affect all benchmark items equally. Straightforwardly harmful outputs may remain harmful under most wrappers, whereas ambiguous, mixed-quality, or stylistically polished outputs can change category depending on phrasing. Because deployment certification often hinges on difficult edge cases—e.g., borderline biosecurity assistance, partial cyber enablement, emotionally manipulative persuasion, or policy-evasive compliance—the operational variance may be larger than aggregate average metrics suggest. Aggregate agreement can remain high while the specific high-risk subset that matters most for release decisions is unstable. This is one reason why headline judge-human agreement rates, even when respectable, are insufficient statistics for certification use (31, 65).

The empirical literature also suggests a deeper problem with retrospective reproducibility. If a benchmark paper from one quarter used a judge prompt that is not fully disclosed, and a later paper reports a new score under a different hidden wrapper, then even exact dataset replication cannot recover whether the score delta reflects model improvement or evaluator drift. This is not an abstract archival concern. Safety claims about progress, plateauing, or regression depend on being able to separate model change from measurement change. Prompt volatility undermines that decomposition unless evaluation artifacts are version-locked and archived.

For deployment certification processes, therefore, the relevant empirical lesson is not simply "judges are imperfect." It is that current judge pipelines often fail a standard measurement requirement that would be routine in other high-stakes domains: *sensitivity analysis to plausible operator choices*. A safety benchmark score should not be accepted as certification-grade evidence unless the judge has been stress-tested across semantically equivalent prompt variants, disclosed wrappers, and representative edge cases, with score intervals reported across those perturbations. The latest robustness-focused papers collectively indicate that this is not yet standard practice (28, 31, 65).

Order effects and pairwise reversals as a source of leaderboard non-determinism

The previous report discussed position bias conceptually; this section instead concentrates on recent quantitative evidence showing how often judgments *reverse* under order swaps and what that implies for pairwise safety ranking systems, arena-style evaluation, and comparative refusal benchmarking.

A particularly striking finding comes from **"Diagnosing Bias and Instability in LLM Evaluation: A Scalable Pairwise Meta-Evaluator."** Across more than **3,600 pairwise comparisons** spanning five instruction-tuned open-weight models and ten open-ended prompts, the authors report that evaluator agreement across judges was uniformly high, even reaching **100% consistency across judges** in the sense of all evaluators rendering the same verdict on a given ordering. Yet **48.4% of verdicts reversed** when the response order was mirrored, i.e., when A–B became B–A (32). This combination—high inter-judge agreement and high mirror-order reversal—is especially important. It shows that consensus among judges does not imply validity; multiple judges can agree on the same serialization artifact.

The result is methodologically devastating for any evaluation regime that equates pairwise vote counts with a stable latent ranking. If nearly half of decisions change when only the display order is reversed, then raw win rates are partly estimating model quality and partly estimating exposure to ordering effects. For safety benchmarking, this problem is amplified because pairwise judgments are frequently used to compare refusal strategies, harmlessness style, red-team response quality, or policy compliance explanations. A model that tends to produce longer or more front-loaded responses may benefit disproportionately when shown first; another may benefit when shown second if the judge implicitly treats the second answer as a refinement. Either way, the leaderboard becomes contaminated by presentational asymmetry (32).

The same study also reports that model rankings varied substantially across prompts by Kendall's Tau analysis, indicating that rank stability is semantically conditional rather than global. This interacts with order effects in an important way. If ordering bias were constant, one might attempt to estimate and subtract it. But the evidence suggests context dependence: some prompts preserve global trends under mirrored order while others produce reversed orders. Thus, a simple universal correction factor is unlikely to suffice. In safety evaluations, where prompts vary widely across domains—self-harm, cyber misuse, extremism, fraud, medical harm, and so forth—the magnitude and direction of ordering bias may differ by hazard class (32).

This has direct implications for comparative safety leaderboards. Many public-facing systems summarize pairwise results into Elo-like ratings or aggregate win rates. Such methods assume that

each comparison is a noisy but informative sample of a stable underlying preference relation. Strong mirror-order reversals violate that assumption. Formally, if the preference relation is not antisymmetric under presentation reversal, then the inferred rating embeds interface effects. In practical terms, a model can gain or lose leaderboard position because of candidate placement conventions, batching logic, or hidden front-end policies rather than any substantive safety improvement.

Table II presents the core empirical pattern and its implications.

Pairwise evaluation property	**Observed empirical pattern**	**Why it matters for safety assessment**
Cross-judge agreement	Can be very high on a fixed ordering	Agreement may reflect shared bias rather than correctness
Mirror-order stability	Nearly half of verdicts reversed in one reported study	Comparative rankings are not invariant to serialization
Prompt-conditional rank stability	Rankings vary by prompt semantics	“Overall safest model” may be an artifact of prompt mix
Global win-rate interpretability	Undermined by order-sensitive local outcomes	Leaderboard ratings can overstate generality
Thresholded pairwise selection	Borderline cases can flip by ordering alone	Deployment gates may depend on UI/serialization choices

One might object that order effects are less relevant for scalar single-response classification tasks, such as harmful/not harmful labeling. However, safety evaluation increasingly relies on pairwise or comparative formats because they are thought to yield higher fidelity or lower variance than direct scoring. For example, evaluators may ask which of two responses is safer, more policy-compliant, or less enabling of harm. The empirical evidence shows that this apparent gain in discriminative sharpness can come at the cost of a new and substantial source of instability. Comparative formats do not eliminate judgment error; they redistribute it into pairwise presentation artifacts.

There is also a subtle policy issue. Suppose a regulator or internal governance board requires evidence that Model X is safer than Model Y on an adversarial harmfulness benchmark. If the supporting evidence consists of pairwise LLM-judge comparisons without mirrored-order controls, then the evidentiary basis is weaker than it appears. The claimed superiority may be true only for one serialization convention. Because model providers and benchmark operators often control that

convention, the absence of mandatory order randomization creates room for accidental or strategic leaderboard shaping.

Another underappreciated aspect is the interaction between order effects and refusal style. Safety-tuned models often produce formulaic disclaimers, moral framing, or prefatory warning sentences. If judges overweight initial tone, the response shown first may anchor the evaluation. Conversely, when the second answer follows a harmful first answer, the judge may interpret the safer response as a correction and reward it disproportionately. Thus, ordering bias is not merely a generic interface problem; it may be structurally amplified in safety tasks because of the rhetorical asymmetry between refusal-like and compliance-like outputs. A concise refusal may look weak when presented first against a longer, polished but subtly leaky answer; the same refusal may look strong when shown second after the leakage becomes salient.

The empirical record also warns against overreliance on multi-judge consensus as a fix. The pairwise meta-evaluator study demonstrates that high evaluator agreement can coexist with dramatic order sensitivity (32). This means an ensemble that uses several judges but preserves a single ordering convention may simply produce a more stable estimate of the same bias. For ensembles to mitigate order effects, they must either include mirrored comparisons explicitly or aggregate over randomized serializations. Otherwise, “consensus” is not evidence against positional artifacts.

The *Know Thy Judge* meta-evaluation perspective reinforces this point by emphasizing robustness meta-evaluation rather than nominal benchmark success. A safety judge should not be judged only by agreement with a validation set; it should be assessed on whether its outputs remain invariant under innocuous transformations such as order reversal when the underlying content relation has not changed. In this view, mirror-order consistency is not a convenience metric but a basic validity criterion for comparative evaluation (31).

From a measurement-theoretic perspective, order instability implies that pairwise judgments in current systems often violate **exchangeability**: swapping labels A and B while preserving content should not change the preferred candidate except for random noise. When reversal rates approach half of all comparisons, the deviation is too large to be dismissed as residual sampling variance. It indicates that the judge is using non-content features as inputs to the decision rule.

For safety certification, this matters in at least three ways.

First, **relative claims become fragile**. Saying that one model is safer than another is much less defensible if the relation flips under mirrored presentation.

Second, **small benchmark deltas become uninterpretable**. If order-induced variance is comparable to or larger than the observed difference between models, the ranking should not drive procurement, release, or regulatory decisions.

Third, **adversarial exploitation becomes easier**. Once the evaluation pipeline has a known positional preference, model providers can shape outputs to benefit from whichever slot they are likely to occupy, or benchmark operators can unintentionally bias results via fixed serialization logic.

A practical reporting implication follows: every safety benchmark using pairwise LaaJ should publish, at minimum, (i) mirrored-order reversal rate, (ii) score/rank correlation between original and swapped orderings, (iii) randomized-order aggregate results, and (iv) hazard-category-specific reversal breakdowns. Without these diagnostics, pairwise safety leaderboards are not adequately auditable.

This is especially true for benchmark suites that are later cited in policy or procurement settings. A regulator reviewing a safety case should ask not simply whether the evaluation used an automated judge, but whether the reported superiority margins survive candidate-order randomization. The latest empirical evidence indicates that this is a nontrivial requirement rather than a box-checking exercise (32, 31).

Style, lexical surface form, and rubric phrasing as hidden score multipliers

This section is distinct from the earlier discussion of "style-over-substance" as a generic vulnerability. The emphasis here is narrower and more empirical: how benchmark scores move when the *same underlying answer* is paraphrased, reformatted, embellished, or evaluated under different rubric wordings, and how such movements distort safety rankings and compliance claims.

Recent studies converge on the view that safety judges are highly sensitive to *surface realization*. The pairwise meta-evaluator framework explicitly includes **surface-level perturbations such as paraphrasing and lexical noise**, alongside mirrored ordering and prompt injection, precisely to diagnose whether verdicts are robust to non-substantive textual changes. The motivation is straightforward: if harmlessness or compliance scores change materially under paraphrase, then the evaluation is partially tracking rhetorical packaging rather than semantic content. The reported findings emphasize that semantic context and structural conditions produce substantial variation in rankings, reinforcing the broader concern that global benchmark scores can obscure local instability (32).

The significance for AI safety is unusually high because "safe" outputs often share a recognizable discourse style: cautionary language, apologies, explicit concern for wellbeing, references to policy,

and offers of benign alternatives. These stylistic markers are, in many cases, correlated with genuine safety behavior. But correlation is not equivalence. A judge that overweights them can be gamed by responses that *sound aligned* while still leaking hazardous information or failing to refuse appropriately. Conversely, a terse but effective refusal can be underrated if it lacks the expected alignment register.

Rubric wording can magnify this effect rather than dampen it. On paper, rubrics should make evaluation more objective by decomposing the task into criteria. In practice, recent evidence suggests that rubrics themselves are an attack and instability surface. The 2026 SoK explicitly cites **"Rubrics as an attack surface: stealthy preference drift in LLM judges"** as part of the threat landscape, which is revealing. It indicates that rubric changes are not merely measurement tweaks but possible channels for inducing directional preference shifts in judge behavior (65). This directly bears on benchmark governance: when a benchmark owner revises the rubric, they may unknowingly alter the score function in a way that rewards a different response style or normative stance.

The distinction between *content-preserving paraphrase* and *criterion-reframing rubric change* is analytically useful:

- **Content-preserving paraphrase instability** means the same answer receives different judgments when reworded.
- **Criterion-reframing instability** means the same answer receives different judgments because the rubric now foregrounds different evaluative dimensions.

Both are present in current systems, and both matter for safety benchmarking.

A recurrent pattern in recent work is that evaluator consistency on unperturbed samples can look acceptable while robustness under surface perturbation degrades materially. This suggests that static benchmark agreement is an incomplete validation target. If a judge is used to score model outputs in a live development loop, developers will naturally optimize not only substance but also presentation. Any stable stylistic cue that the judge rewards becomes part of the optimization target. Over time, the benchmark ceases to incentivize safer behavior and instead incentivizes "judge-legible alignment style."

Table III organizes the main score-distorting channels arising from style and rubric manipulation.

Perturbation type	**Example**	**Mechanism of score change**	**Safety-specific risk**
Paraphrase			

Perturbation type	Example	Mechanism of score change	Safety-specific risk
	Same refusal rewritten more politely or formally	Judge maps tone to safety/compliance	Models optimize rhetorical safety cues
Lexical noise / formatting	Bullet points, headers, emojis, capitalization, spacing	Non-semantic cues alter salience or perceived professionalism	Score depends on formatting rather than hazard content
Verbosity expansion	Same answer with added caveats and alternatives	Length creates appearance of care or completeness	Leakage can hide inside "responsible" prose
Rubric term substitution	"Safe" replaced by "non-harmful," "responsible," or "policy-aligned"	Judge interprets criterion differently	Benchmarks silently change construct
Criterion weighting drift	More emphasis on tone, empathy, or user support	Style dimensions dominate hazard dimensions	Safer-sounding answers beat actually safer ones

The phrase "stealthy preference drift" is apt because rubric changes can be locally plausible yet globally consequential. Consider a rubric revision that adds an item such as "maintains a respectful and empathetic tone" or rephrases "refuses dangerous requests" as "responds responsibly and supportively." Such changes are often defensible and may improve user experience. But they also create opportunities for double-counting alignment style. A model can gain score by adding affective framing, moral language, or polished structure even if its substantive safety behavior is unchanged. If that gain then feeds into a public leaderboard or a release gate, the benchmark effectively rewards style engineering.

The 2025 robustness paper's basic thesis—that LLM judges are inherently vulnerable to adversarial attacks and subtle perturbations that manipulate outcomes—should be interpreted broadly enough to include these style-level manipulations. Not every exploit needs to look like an overt prompt injection. In many safety-evaluation settings, a "soft" exploit that induces favorable rubric matching through harmless-seeming phrasing may be sufficient to alter the result (28).

Another reason this issue is hard to detect is that style-induced inflation may improve nominal human agreement on some benchmark distributions. If human annotators themselves are influenced by tone, disclaimer presence, or perceived professionalism, a judge that learns these cues can appear

well-calibrated on in-distribution validation sets. The failure only becomes visible when the benchmark encounters adversarially polished unsafe outputs, intentionally terse but safe outputs, or cross-domain style shifts. This suggests that robustness meta-evaluation should include *counter-stereotypical* examples: substantively safe but stylistically plain responses, and unsafe or policy-violating answers wrapped in high-status compliance rhetoric.

The implications for regulatory compliance are substantial. Claims such as "the model complies with the safety policy in 97% of harmfulness scenarios" are stronger than the underlying measurement often warrants if the compliance score is substantially sensitive to wording and presentation. In regulated environments, one would normally ask whether the measurement instrument is invariant under paraphrase and whether revisions to the scoring rubric have been validated as non-material. Current LLM-judge practice rarely meets that standard.

This is particularly problematic when benchmarks compare models from different families with distinct default discourse styles. Some models are chatty, hedged, and highly apologetic; others are concise and instruction-focused. If the judge's rubric or prompt implicitly favors the former style, then cross-family evaluation becomes confounded by house style. The benchmark may then report a safety difference that is partly a stylistic family effect rather than a true difference in hazardous capability management.

The literature also indicates that style sensitivity and rubric sensitivity can interact with attack methods. The 2026 SoK lists one-token, emoji-based, optimization-based prompt injection, and rubric attack work in the LaaJ context (65). These are not isolated curiosities. They demonstrate that the judge's scoring function often has high local sensitivity to small lexical features. Once that is true, it is unrealistic to assume that ordinary benchmark paraphrases are measurement-invariant. The same vulnerability surface that makes an emoji attack possible also makes "safe-sounding" polishing consequential.

For benchmark designers, the lesson is that rubric clarity alone is insufficient. A rubric must be empirically tested for **surface-form invariance**. This requires evaluating whether semantically equivalent paraphrases of candidate outputs and semantically equivalent rewrites of rubric items leave judgments materially unchanged. If not, the rubric is not merely imprecise; it is operationally unstable.

A practical auditing protocol would include:

- paraphrase sets for candidate responses;
- lexical and formatting perturbation sweeps;
- style-stratified performance reporting;
- ablation of affective/hedging language;

- and rubric-version comparison on a locked challenge set.

Without such controls, benchmark owners cannot know whether observed score improvements reflect safer models or more judge-compatible prose.

The current state of evidence therefore supports a strong claim: many present-day safety benchmark scores are partly *style scores in disguise*. The problem is not that style never matters. In some settings, tone and framing are indeed components of safe behavior. The problem is that current LaaJ systems often fail to separate style as a legitimate subcriterion from style as a confounder that overwhelms the actual hazard-management objective. Recent robustness and SoK work indicates that this separation remains unresolved (28, 65, 32).

Cross-family disagreement, judge replacement, and model/version drift in longitudinal safety tracking

This section differs from the previous report's discussion of shared-model blind spots and preference leakage. The emphasis here is on *empirical instability over time and across judge identities*: what happens to safety benchmark scores when evaluators are swapped across model families or when a nominally "same" judge changes version, policy stack, or hidden system behavior.

The first issue is **cross-family non-equivalence**. The *Know Thy Judge* paper frames the problem at the level of safety judge meta-evaluation: multiple LLM-based safety judges exist, each claiming stronger performance, yet they are released under inconsistent validation standards and robustness disclosures. This means benchmark outcomes are not simply sensitive to prompt design but also to the *choice of judge family itself* (31). In practical terms, replacing one safety judge with another can alter the benchmark's latent normative function. Even when both judges are instructed with the same rubric, they may differ in severity, deference to explicit policy wording, tolerance for ambiguity, and susceptibility to stylistic cues.

This is not surprising from a modeling perspective. Large language models differ in pretraining data, instruction tuning, post-training alignment, refusal norms, and moderation overlays. A safety judge built on one family may systematically penalize directness, while another may emphasize user support or policy formalism. The resulting disagreement is not necessarily random error; it can reflect stable but divergent evaluative priors. For cross-family evaluation, this means that "safety score" is not a scalar independent of the evaluator. It is a function indexed by judge family.

The second issue is **version drift**. Many deployed judges are API-mediated or periodically updated. Even when the model name remains constant, the effective evaluation system can change due to checkpoint refreshes, system prompt revisions, moderation stack updates, context-window handling

changes, decoding defaults, or undisclosed inference optimizations. The earlier report already noted the existence of silent model refresh as a threat surface. The empirical question pursued here is what that means for safety metrics over time.

The answer, based on the current literature, is that longitudinal score tracking is less stable than it appears. Because judge outputs are already sensitive to prompt and rubric perturbation, any untracked version change can propagate into benchmark movement that is difficult to distinguish from genuine model improvement or regression. A model developer might report that a new release improved safety benchmark performance by several points. Unless the judge version is frozen and auditable, that delta is confounded. It may reflect some combination of model change, judge drift, and interaction between the two.

The 2026 SoK's taxonomy of LaaJ security threats reinforces this concern by explicitly including both training-time and inference-time vectors, as well as backdoor and preference-drift attacks, within the judge threat landscape (65). A backdoored or otherwise drifted judge need not catastrophically fail to invalidate longitudinal comparisons; even subtle shifts in severity or criterion weighting can move a thresholded safety metric.

A useful distinction is between:

- **observable version drift**, where the provider announces a new judge release;
- **silent version drift**, where outputs change without explicit versioning;
- **functional drift under constant label**, where the "same" judge name masks nontrivial behavioral change.

For certification and leaderboard governance, the third is particularly problematic because it undermines reproducibility while preserving the appearance of continuity.

Table IV outlines major sources of drift and their benchmark consequences.

Drift source	**What changes**	**Typical observed consequence**	**Governance consequence**
Judge family swap	Different base model/ alignment stack	Rank order and threshold changes	Cross-study scores become non-comparable
API model update	Hidden checkpoint or policy refresh	Longitudinal benchmark movement	Audits cannot reproduce prior results

Drift source	What changes	Typical observed consequence	Governance consequence
System prompt revision	Different hidden instructions or examples	Severity or rationale shifts	Benchmark owner loses control of metric definition
Moderation-layer change	Upstream filtering or safety wrapper adjustment	More conservative or inconsistent scoring	Apparent safety trend may be evaluator artifact
Decoding/ configuration change	Temperature, reasoning mode, max tokens	Variance or verbosity bias shifts	Measurement error changes over time

Cross-family disagreement also complicates efforts to claim external validity. Suppose an internal safety team uses Judge A to certify that a model satisfies a hazardous-content threshold, while an external auditor or regulator uses Judge B. If the two judges implement different latent preferences, they may produce incompatible verdicts on the same benchmark suite. This is not merely an inconvenience; it undermines the portability of compliance claims. In effect, the provider is certified against one measurement regime, not against a universally accepted construct.

The pairwise meta-evaluator study indirectly supports this point by showing that rankings vary substantially across prompts and conditions (32). If within a single framework rankings are already prompt-conditional, then across judge families they are even less likely to be invariant. Different judges may react differently to the same prompt mix, leading to family-specific leaderboard topologies.

There is also an underexplored interaction between version drift and adversarial optimization. Developers optimize their models against whatever evaluator is currently in use. If the judge changes over time, the optimized response style may no longer be rewarded, or new exploitable artifacts may appear. This means benchmark drift can create the illusion of progress or regression simply by altering what the optimization loop has learned to target. A model that was highly "judge-compatible" under version (v*1) may lose score under (v*2) even without becoming less safe in any human-relevant sense.

In regulated settings, such instability clashes with normal expectations for conformity assessment. A certification instrument should be calibrated, version-controlled, and traceable. By contrast, many LLM judges are ephemeral software services with incomplete disclosure and limited archival

reproducibility. The empirical robustness literature now makes clear that this is not a minor administrative defect. Because score sensitivity is substantial, weak version control directly degrades evidentiary quality.

The *Know Thy Judge* paper is especially relevant because it reframes judge development as requiring *meta-evaluation* comparable in seriousness to evaluation of the target models themselves. That is, one should not trust a safety judge merely because it performs well on a reference dataset; one should ask how robust it is across conditions, how large its validation set is, what hazards it was tested on, and how much transparency exists regarding its evaluation protocol (31). Applied to drift, this means every substantive judge update should trigger re-validation rather than inheriting prior credibility by name alone.

A further implication concerns public leaderboards. If leaderboard maintainers replace judges, update API versions, or alter hidden prompts without re-baselining prior submissions, the leaderboard becomes temporally incoherent. Scores from different months may not live on the same scale. This is common enough in fast-moving LLM ecosystems that a static numerical ordering can give a false impression of precision.

For deployment certification, one cannot simply require "evaluation by an LLM judge." One must specify *which* judge, *which version*, *which wrapper*, and *which frozen execution environment*. More importantly, one must show that the result is not idiosyncratic to that choice by conducting cross-judge sensitivity analysis. A certification claim supported only by a single judge family and a single version is better thought of as an internal screening result than as robust external evidence.

The current evidence therefore supports several strong operational claims:

1. **Cross-family judge replacement can materially change measured safety.**
2. **Version drift can mimic model progress or regression.**
3. **Undisclosed evaluator changes are a direct threat to leaderboard and certification validity.**
4. **Longitudinal safety trendlines without frozen judges are scientifically weak.**

These claims follow not from one isolated anomaly but from the convergence of robustness evaluations, meta-evaluation critiques, and the broader LaaJ security literature (28, 31, 65).

Implications for benchmark governance and the evidentiary role of mitigations

This final section is different from the earlier report's general governance remarks. Here the focus is on what the empirical instability findings imply for the *use* of safety benchmark scores in

leaderboards, regulatory submissions, and deployment gates, and on how proposed mitigations should be evaluated in light of the evidence.

A first-order implication is that many current AI safety leaderboards should be treated as **screening instruments, not definitive measurements**. The empirical findings surveyed above—prompt sensitivity, order reversals, surface-form dependence, rubric drift, and cross-judge non-equ

Cross-Model and Cross-Family Evaluation Reliability in Safety-Critical LLM-as-a-Judge Pipelines

Human alignment figures are best interpreted as conditional validity, not portable proof of evaluator trustworthiness

A recurring claim in the LLM-as-a-Judge literature is that strong proprietary judges reach "human-level" or "human-comparable" agreement. The most widely repeated numeric range remains roughly 80% agreement with human preferences for strong frontier judges, including claims summarized in practitioner-facing reviews and framework papers that cite earlier GPT-4-based work as achieving over 80% agreement with humans in pairwise preference settings. Yet, for safety assessment, these numbers are better treated as *conditional validity under a particular annotation regime* than as a general warrant for deployment as a certification instrument. The crucial issue is not whether a judge can match humans on some held-out comparisons, but whether that agreement remains stable across domains, across evaluative constructs, across candidate model families, and across adversarially relevant output distributions. The newer evaluation discourse increasingly frames judges as *measurement instruments* whose reliability and uncertainty must themselves be estimated, rather than as neutral oracles whose outputs can be treated as direct ground truth (1).

This shift matters especially in AI safety assessment because the underlying construct is usually not simple user preference. Safety evaluation often blends rule compliance, harm minimization, factual sufficiency, calibrated refusal, proportional assistance, fairness, and deception resistance. A judge validated primarily on generic helpfulness or pairwise preference data can therefore exhibit deceptively high "agreement with humans" while still underperforming on the specific latent construct regulators or benchmark designers care about. Recent commentary in the evaluation community argues that benchmark plateaus may partly reflect measurement limits, including judge error and instrument saturation rather than true capability ceilings, which directly undermines the common practice of reading small leaderboard differences as substantive evidence of safety superiority (1).

The distinction between *agreement with humans* and *agreement with the intended safety construct* is analytically decisive. A judge may agree with crowdworkers on superficial preference cues, yet disagree with domain specialists on whether a response creates downstream misuse risk. Conversely, a safety-conservative judge may score responses more harshly than non-expert annotators but better approximate an institutional safety policy. Thus, raw agreement percentages should not be interpreted as universally transferable properties of a judge family. They are joint functions of:
1) the reference human population,
2) the task format,
3) the rubric,
4) the candidate response distribution, and
5) the aggregation method used to convert judgments into scores.

The practical literature now supports this more restrictive interpretation. In a recent empirical study of practical drop-in methods for improving LLM judges on RewardBench 2, the reported baseline GPT-5.4 judge accuracy was 71.7%, which rose to 83.6% when task-specific criteria injection and ensembling were combined. This is significant for two reasons. First, it shows that a double-digit performance swing can come from evaluator-side scaffolding alone rather than any change in the underlying candidate models. Second, it implies that a headline "human agreement" value should not be attributed solely to model capability; it is partly a property of the *evaluation protocol* wrapped around the judge (2). In other words, cross-paper comparisons of judge reliability can be substantially confounded when one study uses a bare prompt and another uses criteria injection, self-consistency, or multi-call ensembling.

The domain-dependence point is also explicit in practitioner guidance. One 2026 operational review states that agreement typically drops by 10–15% in specialized fields, recommending use as a screening tool rather than a final decision-maker in such cases. Although that source is not itself a primary research paper, it usefully reflects a broad empirical pattern already visible across safety and expert-domain evaluation: the more the construct depends on specialized factual, legal, biomedical, or policy interpretation, the less defensible it becomes to transfer generic human-agreement claims into assurance statements (3). For safety benchmarks, this implies that evaluations involving cyber misuse, biosecurity assistance, self-harm intervention, discrimination, or regulatory compliance should not inherit confidence from generic preference benchmarks without construct-specific validation.

The same concern arises in papers proposing joint safety-evaluation frameworks. For example, the Jo.E framework cites prior work claiming GPT-4 judges achieve over 80% agreement with human preferences while also acknowledging systematic biases such as position bias, verbosity bias, and self-enhancement bias. The implication is subtle but important: a judge can attain strong average

agreement while still exhibiting structured, directional distortions that matter for benchmark comparability. A high average correlation is therefore compatible with materially incorrect rankings for certain subsets of items—particularly those where safety hinges on concise but correct refusal, partial compliance, or trade-offs between informativeness and caution (4).

A measurement-science framing helps clarify why this matters. Human agreement is often reported as if it were analogous to accuracy against a stable label. In reality, safety judgment more closely resembles latent trait estimation under noisy, imperfect raters. If the human reference set is itself heterogeneous, then "agreement with humans" partly reflects the compatibility between the judge's normative priors and those of the annotation pool. This produces at least four interpretive hazards:

Hazard	Description	Why it matters for safety leaderboards
Reference-population dependence	Agreement changes with annotator expertise and policy orientation	A leaderboard may reflect crowd norms, not institutional safety standards
Construct slippage	Preference labels differ from safety labels	Scores may reward likability over risk minimization
Distribution mismatch	Validation data differ from deployment prompts	Human-aligned judges can fail on adversarial or specialized items
Protocol confounding	Prompting, rubric injection, and aggregation alter reliability	Reported judge quality is not portable across labs

The recent turn toward reliability estimation and uncertainty quantification directly addresses these hazards. The February 2026 AI Evaluation Digest highlights methods that estimate judge reliability, attach uncertainty to preference-based rankings, and audit judges psychometrically as instruments. This is especially salient for cross-model and cross-family comparisons because it implies that a benchmark score should be reported with measurement uncertainty not only from finite test-set size, but also from evaluator uncertainty and evaluator-specific variance (1). In a mature safety-assurance setting, one would expect at minimum: confidence intervals over model scores, sensitivity analyses over judge families, and evidence that the measured ordering is robust to alternative credible adjudicators.

Another underappreciated issue is that strong human agreement in aggregate can conceal *conditional divergence by response type*. The newly reported comparison of holistic versus atomic judges on QA-style benchmarks found that the holistic judge's advantage was concentrated in

“partially_supported” cases—i.e., incompleteness detection—while TruthfulQA showed a small atomic edge. This is not directly a cross-family result, but it illustrates a broader point: evaluator performance is often heteroskedastic across item types. Safety datasets have analogous strata, such as overtly unsafe requests, benign requests with embedded risky context, subtle policy circumvention, or responses that are factually correct but normatively under-refusing. A judge that performs well overall may still fail systematically on the most safety-relevant subsets (5).

For cross-model reliability, the principal lesson is therefore not that human agreement numbers are meaningless, but that they are *insufficiently specific* for high-stakes claims. A judge showing 80%+ agreement under one protocol does not imply that:

- it is equally reliable across model families,
- it preserves rankings under safety-specific rubrics,
- it remains robust under adversarially optimized candidate outputs, or
- its measured agreement translates into low false-negative rates on harmful behaviors.

This has immediate implications for compliance and certification. If a developer claims that its safety benchmark was “validated against human judgment,” the evidentiary question should be: *which humans, on what tasks, under which rubric, against what failure distribution, and with what uncertainty?* Without those details, agreement figures function more as marketing shorthand than as scientific support. In the context of deployment certification, the minimum defensible standard should move from “judge correlates with humans” to “judge-family-conditioned score intervals and error profiles are characterized for the claimed deployment domain” (1).

A related implication concerns benchmark transportability across institutions. Suppose Lab A validates a judge on open-ended preference tasks and uses it to score harmful-content refusals, while Lab B uses a different judge family validated on policy-compliance annotations. Both labs may report strong human agreement, yet their scores are not necessarily commensurable. The disagreement lies not only in model identity but in the *implicit construct encoded by the validation pipeline*. This is one reason why safety leaderboard comparisons across organizations remain fragile even when each organization independently reports respectable judge quality.

The latest evidence on evaluator improvement also cuts in two directions. On one hand, criteria injection and ensembling materially improve RewardBench 2 accuracy, showing that evaluator reliability can be engineered upward without fine-tuning (2). On the other hand, this very malleability means that reported benchmark results are sensitive to protocol design choices invisible to casual readers. If one vendor uses a single-call judge and another uses k-way ensembles with task-specific criteria, the resulting scores are not directly comparable even if both claim to be using

“the same base judge model.” Evaluation reliability is therefore inseparable from evaluation governance.

For this reason, safety-oriented reporting should distinguish at least three layers of validity:

Layer	Question	Typical evidence	Limitation if reported alone
Face validity	Does the rubric look reasonable?	Qualitative examples	Insufficient for reliability claims
Human correspondence	Does the judge agree with annotators?	Accuracy, kappa, correlation	Can hide construct mismatch and stratified failures
Decision validity	Does the evaluation support the deployment decision it is used for?	Error analysis on policy-relevant cases, uncertainty, cross-family sensitivity	Rarely reported, but essential for certification

The emerging literature on reliability estimation suggests that the field is starting to recognize this hierarchy, but current safety benchmark practice remains behind what measurement theory would recommend. Especially for cross-model and cross-family evaluation, “agreement with humans” should be treated as a useful but narrow diagnostic, not as a sufficient basis for score portability, leaderboard legitimacy, or regulatory assurance (1).

Judge-family divergence should be modeled as structured measurement variance rather than dismissed as noise

While previous sections in earlier subtopic reports discussed cross-family disagreement as a generic source of instability, this section focuses more narrowly on *how divergence across evaluator families alters the interpretation of comparative safety claims*. The central point is that disagreement among judge families is not merely random annotation noise; it often reflects systematic differences in evaluative priors, calibration, failure sensitivity, and thresholding behavior. As a consequence, cross-family evaluation reliability must be analyzed as a problem of *instrument non-equivalence*.

The Jo.E framework offers one of the more concrete recent windows into this issue by reporting pairwise evaluator agreement among GPT-4o, Claude 3.5 Sonnet, and Llama 3.1. On their benchmark, GPT-4o ↔ Claude achieved Cohen’s kappa of 0.71 and Pearson correlation of 0.78;

GPT-4o ↔ Llama achieved kappa 0.64 and correlation 0.69; Claude ↔ Llama achieved kappa 0.67 and correlation 0.73; and all three together reached Fleiss' kappa 0.68. These are substantial but far from interchangeable levels of agreement. In classical measurement terms, they indicate that the evaluators are related enough to detect overlapping patterns, yet not so interchangeable that one can substitute for another without affecting the resulting score distribution (6).

The significance of these values is often misunderstood. A kappa around 0.64–0.71 may be described as moderate-to-substantial agreement, but for benchmark comparability the relevant question is not merely whether agreement is "good." The more operational question is whether the residual disagreement is large enough to change rank orderings, pass/fail outcomes, or claims of improvement. In safety evaluations, those consequences are highly plausible. When score gaps between candidate models are only a few percentage points, cross-family judge divergence of this magnitude can easily dominate the substantive differences under study. Thus, reporting a single leaderboard without judge-family sensitivity analysis implicitly assumes a stronger degree of interchangeability than the data warrant.

The same Jo.E results are useful because they also reveal a pattern of *graded divergence* rather than uniform disagreement. The highest agreement appears between two leading proprietary aligned models, GPT-4o and Claude 3.5 Sonnet, while agreement is lower when either is paired with Llama 3.1. The authors interpret this as evidence that models with similar post-training philosophies may share more evaluative behavior, whereas open-weight models trained with different safety stacks contribute useful diversity (7). For cross-family reliability analysis, this supports the hypothesis that the judge-family effect is not random heterogeneity but reflects ecosystem structure: models closer in training objectives and safety norms tend to agree more, which may increase consensus but also increase correlated blind spots.

This has a direct implication for benchmark design. If the goal is stable internal scoring for a single organization, using judges from similar families may improve consistency. If the goal is robust external assurance, however, that same similarity may reduce the ability to detect family-specific over-scoring or under-recognition of certain harms. Reliability for governance is therefore not identical to reliability for convenience. A highly internally consistent panel may still be epistemically narrow.

A useful way to formalize this is to decompose observed benchmark variance into at least four components:

$$\text{Observed score variance} = \text{candidate-model variance} + \text{item variance} + \text{judge-family variance} + \text{interaction variance}$$

The underreported term is the interaction between candidate model and judge family. That interaction captures cases where one evaluated model is systematically favored by one judge family more than another. This is especially important in same-family or same-ecosystem evaluation, where stylistic familiarity, refusal templates, policy wording, or latent preference overlap can create a family-specific uplift. Existing reports already noted the conceptual risk of shared priors; the new point here is that for benchmark comparability, the relevant quantity is not just "some disagreement exists," but *how much of the score difference is attributable to judge-family interactions rather than target-model safety*.

The practical consequences can be summarized as follows:

Scenario	**What is often assumed**	**What cross-family evidence suggests**
Same benchmark scored by different judge families	Scores are roughly comparable after normalization	Rank order and pass/fail thresholds may shift materially
High pairwise kappa among judges	Evaluators are interchangeable	Agreement may still be insufficient for certification-grade substitution
Similar proprietary judges agree strongly	Convergence indicates validity	It may also indicate shared norms and correlated omissions
Open-weight judge disagrees more often	It is lower quality	It may be surfacing different but policy-relevant failure modes

This last point is particularly important because disagreement is commonly pathologized. In measurement contexts, lower agreement is not always bad; it can be informative if it reveals sensitivity to previously missed dimensions. The Jo.E authors explicitly treat moderate agreement with low joint failure as a "sweet spot," suggesting that some evaluator diversity is beneficial because it catches blind spots missed by more similar models (7). That framing is persuasive for safety evaluation: a perfectly homogeneous evaluator set may look reliable while systematically overlooking the same hazard classes.

However, this does not mean any disagreement is helpful. There is a difference between *constructive diversity* and *uncontrolled non-equivalence*. Without human adjudication or a meta-evaluation benchmark, cross-family divergence cannot automatically be interpreted as either healthy pluralism or error. This is why future evaluator reporting should separate:
1) average inter-judge agreement,

2) family-conditioned disagreement by category, and
3) adjudicated error direction on disputed items.

The newly noted movement toward uncertainty-aware ranking methods is relevant here. If rank confidence intervals explicitly incorporate evaluator uncertainty, then a leaderboard can represent not just a point ordering but a range of plausible orderings under reasonable judge replacements (1). Such reporting would be far more honest for safety applications, where single-number scores are frequently overinterpreted.

A further challenge is that cross-family divergence is often benchmark-specific. The holistic-versus-atomic judge study on ASQA, QAMPARI, and TruthfulQA found benchmark-dependent patterns: holistic judging matched or exceeded atomic judging on two benchmarks while atomic had a small edge on TruthfulQA, with sensitivity concentrated in incompleteness detection and strong dependence on reference quality (5). Although this study concerns decomposition style rather than model family, it underscores a broader lesson: evaluator differences are not globally ordered. A judge family can outperform another on one benchmark and underperform on another because the benchmark stresses different latent skills. Safety benchmarks should therefore avoid collapsing cross-family divergence into a single global "best judge" narrative.

This also complicates regulatory and procurement claims. If a vendor argues that its safety score was obtained using a "state-of-the-art judge," the relevant response is that state-of-the-art with respect to which construct and benchmark? A judge family optimized for generic preference ranking may not be state-of-the-art for adversarial robustness scoring, refusal adequacy, or policy-conditional compliance assessment. Absent cross-family calibration, such claims risk treating benchmark-specific dominance as universal evaluator validity.

For benchmark governance, the strongest near-term recommendation is to publish *family sensitivity profiles*. At minimum, benchmark maintainers should disclose whether the ordering of leading models is stable across at least one proprietary and one open-weight judge family, ideally with family-conditioned score intervals. This would not eliminate disagreement, but it would make clear whether a claimed safety advantage is robust or merely evaluator-contingent.

A concise way to express the problem is that current leaderboards often present:

$$\text{Model safety score}$$

when what they actually possess is closer to:

$$\text{Model safety score} \mid \text{judge family}, \text{prompt template}, \text{aggregation rule}, \text{rubric}$$

Until that conditionality is made explicit, cross-model and cross-family comparisons will remain vulnerable to overclaiming. The recent reliability discourse suggests the field is moving in the right direction, but most published and operational safety evaluations still understate how much evaluator-family choice shapes the apparent metric itself (1).

Correlated evaluator blind spots become visible when agreement is paired with joint-failure analysis rather than consensus alone

Earlier subtopic reports already treated shared-model blind spots conceptually. The distinct contribution here is to examine how *joint-failure statistics* change the interpretation of agreement and why they are indispensable for cross-family reliability in security-sensitive evaluation. Consensus measures such as accuracy, correlation, or kappa tell us how often judges agree. They do **not** tell us whether judges are jointly wrong on the same safety-critical items. In adversarial environments, the latter quantity is often more important.

The Jo.E framework provides rare explicit numbers on this point. While pairwise evaluator agreement among GPT-4o, Claude 3.5 Sonnet, and Llama 3.1 was moderate to high, the reported joint failure rates were 4.2% for GPT-4o ↔ Claude, 6.8% for GPT-4o ↔ Llama, 5.4% for Claude ↔ Llama, and 2.1% when all three evaluators were combined. The authors describe 332 cases out of 15,847 total where all three evaluators missed a ground-truth vulnerability (6, 7). For safety assessment, that figure is more operationally meaningful than kappa alone because it estimates the residual hazard class that survives evaluator diversity.

This distinction matters because a panel of judges can show attractive agreement while still sharing the same omission pattern. Prior work on order effects and evaluator instability already demonstrated that multiple judges can agree strongly and still reverse verdicts under mirrored orderings, indicating that consensus can form around an artifact rather than around the intended construct. The same logic applies to shared blind spots: agreement is weak evidence of validity unless accompanied by evidence that judges fail independently enough to catch each other’s errors. Joint-failure analysis operationalizes exactly that concern.

From a security perspective, correlated evaluator blind spots matter in at least three ways:

Blind-spot type	Failure pattern	Security relevance
Shared refusal-template familiarity	Judges over-credit familiar cautionary language	Unsafe content may pass if wrapped in alignment-flavored prose

Blind-spot type	Failure pattern	Security relevance
Shared policy priors	Judges miss harms that fall outside common moderation conventions	Novel jailbreaks or ambiguous misuse cases are underdetected
Shared training-distribution bias	Judges fail on rare or domain-specific hazards	Biosecurity, cyber, or legal-risk cases receive inflated safety scores

The reported Jo.E numbers suggest that evaluator diversity reduces but does not eliminate these problems. A 2.1% all-judge miss rate may look small, but in high-volume deployment that can be large in absolute terms, and more importantly, the statistic depends on the benchmark's ground-truth coverage and adversarial depth. If the benchmark underrepresents hard-to-detect harms, the true shared-blind-spot rate may be higher. Thus, joint failure should be read as a lower bound on uncovered risk under the benchmark's sampling assumptions.

The same Jo.E paper provides another important result: adversarial agents contributed unique detections that the LLM evaluators missed. The reported ablation suggests that removing adversarial agents reduced performance by 6.6%, and 23.4% of jailbreaks were caught only by agents rather than by the evaluator ensemble (8). This is a strong empirical indicator that even cross-family evaluator ensembles are not enough by themselves. They reduce correlated blind spots among judges, but they do not replace independent attack-generation mechanisms.

That finding has broader methodological significance. In many current safety pipelines, the same broad class of LLMs is used to:
1) generate candidate responses,
2) generate red-team prompts,
3) judge outputs, and sometimes
4) review appeals.

Even when different vendors are used, these components may still inhabit a common training and alignment ecosystem. Therefore, "cross-model" does not automatically imply "independent." A panel consisting of several mainstream frontier chat models may be diverse in branding and architecture while still correlated in normative priors, moderation style, and lexical heuristics. The joint-failure metric is one of the few available ways to probe whether this apparent diversity yields real error decorrelation.

This also reframes the governance value of open-weight judges. An open-weight model with somewhat lower average agreement but different failure structure may improve the system-level miss rate more than another proprietary model that agrees more often with the first proprietary

judge. The Jo.E agreement pattern is suggestive in exactly this direction: the lower GPT-4o ↔ Llama agreement relative to GPT-4o ↔ Claude may reflect useful diversity rather than simple inferiority (7). For safety assurance, what matters is not only which judge is individually strongest, but which combination minimizes undetected hazard.

This can be formalized via a portfolio perspective. Suppose each judge (J*i) has an individual false-negative rate (FN*i), but those errors are correlated. Then the ensemble false-negative rate depends not merely on the mean (FN_i), but on the covariance structure across judges. Adding a judge that is slightly weaker on average but less correlated with the existing judges can improve the ensemble more than adding a stronger but highly redundant one. This is a familiar result in ensemble learning, but it is especially consequential for AI safety evaluation because benchmark users often optimize for individual-judge agreement rather than ensemble coverage.

The Jo.E paper effectively argues for this diversity logic by emphasizing moderate agreement with low joint failure as the desired balance. That framing is valuable, though still incomplete. The remaining methodological gap is that joint-failure estimates must themselves be stratified by risk category. A 2.1% global miss rate could hide much higher rates on rare but catastrophic classes, such as sophisticated jailbreaks, covert exfiltration assistance, or domain-specific harmful advice. The paper hints at category-specific differences by noting that adversarial agents were particularly important for robustness and jailbreak categories (8). This suggests the next generation of reliability reporting should include *category-conditioned joint miss rates*.

Another reason consensus alone is misleading comes from benchmark incompleteness. The February 2026 digest notes that apparent benchmark plateaus may reflect judge error or measurement limits rather than true capability ceilings (1). If judges saturate on easy items and jointly miss difficult ones, agreement may remain high while progress on the hard tail goes unmeasured. In that setting, the leaderboard can appear stable not because models are equally safe, but because the evaluator ensemble lacks sensitivity where it matters most.

For deployment certification, this means that any claim of "independent evaluation" should specify whether independence was assessed at the level of *error correlation*, not merely vendor diversity. A robust certification pipeline would require at least the following disclosures:

Required disclosure	**Why it matters**
Pairwise agreement among evaluators	Measures gross consistency
Joint false-negative rate against adjudicated vulnerabilities	Measures residual undetected risk

Required disclosure	Why it matters
Category-specific miss rates	Identifies concentration of blind spots
Effect of adding heterogeneous judges	Tests whether diversity is substantive or cosmetic
Agent-only and human-only catch rates	Reveals what the evaluator ensemble still misses

Without these disclosures, cross-family evaluation may provide only a superficial appearance of rigor. A regulator or auditor seeing multiple evaluator names might infer independence that is not empirically established.

There is also an adversarial implication. Once attackers know that benchmark approval relies on a stable set of judge families with partially shared blind spots, they can optimize outputs to exploit those common omissions. This creates a form of *ensemble gaming*: not merely manipulating one judge prompt, but discovering response features that survive the full evaluator panel. The more homogeneous the panel's training and alignment ecosystem, the easier such attacks may become in principle. Joint-failure analysis can at least indicate whether the panel leaves a wide common attack surface.

The practical mitigation is therefore not just "use multiple judges," but "use multiple judges whose residual errors are empirically shown to be sufficiently decorrelated, and supplement them with adversarial generators plus human escalation on contested or high-impact cases." The current evidence supports this layered approach more strongly than it supports any single best-judge paradigm (6, 8).

Benchmark comparability fails when evaluator-induced variance is not separated from model-induced variance

One of the most consequential but underdiscussed effects of cross-model and cross-family reliability problems is their impact on *benchmark comparability*. Safety leaderboards, procurement scorecards, and internal go/no-go gates often presume that the reported metric mainly reflects differences among target models. Yet recent evidence suggests that a nontrivial portion of observed variation can instead arise from evaluator choice, evaluator aggregation, and evaluator-family interactions. When those components are not explicitly estimated, the benchmark becomes a conflation of model behavior and measurement apparatus.

The RewardBench 2 judge-improvement study illustrates the problem starkly. A baseline GPT-5.4 judge achieved 71.7% accuracy, task-specific criteria injection improved this by 3.0 percentage points, ensemble scoring improved it by 9.8 percentage points at 5x cost, and combining the two reached 83.6%—an 11.9-point gain over baseline (2). Those gains do not reflect changes in the candidate models being evaluated; they arise entirely from the *evaluator protocol*. Therefore, if two organizations evaluate the same safety model using the same base judge family but different judgment wrappers, they may obtain meaningfully different benchmark rankings. Any public comparison that ignores this is methodologically incomplete.

This issue is amplified when benchmarks are near saturation. The February 2026 digest explicitly warns that benchmark plateaus may reflect judge error or measurement limits rather than real capability ceilings (1). In saturated regimes, the signal-to-noise ratio for small model improvements is already weak. If evaluator-induced variance is of the same order as the alleged performance difference, then leaderboard positions become unstable and overinterpretable. A one- or two-point "win" may simply be a property of judge configuration rather than a property of the model.

The problem can be stated in a simple identifiability form. Let:

$$ S_{obs} = S_{true} + \epsilon_{judge} + \epsilon_{prompt} + \epsilon_{aggregation} + \epsilon_{interaction} $$

where S_{obs} is the observed benchmark score and S_{true} is the latent model capability the benchmark is intended to measure. If the evaluator terms are not estimated or bounded, then S_{true} cannot be recovered with confidence. Existing benchmark reporting usually provides uncertainty from finite sample size, but rarely from evaluator-family substitution or protocol variance. This yields overly narrow confidence narratives.

The effects are not only statistical but institutional. Safety leaderboards increasingly serve as evidence in three higher-stakes contexts:

Context	How benchmark scores are used	Why comparability matters
Regulatory submissions	To support claims of policy compliance or mitigated risk	Non-comparable scores can misstate legal readiness
Enterprise procurement	To choose model vendors for sensitive deployments	Vendor selection may hinge on evaluator artifact

Context	How benchmark scores are used	Why comparability matters
Internal release decisions	To justify moving models from restricted to wider access	Deployment risk may be underestimated if scores are evaluator-inflated

In all three cases, evaluator-induced variance is not a minor technicality; it affects downstream decisions with real consequences.

The Jo.E paper, though framed as a comprehensive safety framework, indirectly reinforces this point through its multi-evaluator design. It reports differing evaluator agreements and emphasizes the benefit of structured diversity over single-evaluator use (6, 8). The deeper lesson is that a benchmark score produced by one evaluator cannot automatically be placed on the same scale as a score produced by another. Even if both are "good judges," their thresholds, omissions, and response to ambiguity may differ enough to destroy numerical comparability.

This has immediate consequences for longitudinal tracking as well. If benchmark maintainers silently upgrade a judge model, change the system prompt, alter aggregation from single-call to ensemble, or replace one family with another, the resulting time series no longer supports straightforward trend interpretation. A reported improvement in model safety could actually be a measurement drift artifact. Earlier reports already discussed version drift conceptually; the additional point here is that *cross-family reliability and benchmark comparability are the same problem viewed from two different angles*. Across labs, the issue appears as lack of comparability; across time, it appears as lack of measurement invariance.

A particularly serious failure mode arises when benchmark operators normalize scores post hoc and thereby obscure evaluator differences. For example, if each evaluator family is separately calibrated to produce roughly similar score distributions, observers may infer comparability from superficial alignment of means. But mean alignment does not ensure rank preservation, threshold equivalence, or matched error structure. Two evaluators can be distributionally calibrated and still disagree on the most safety-critical tails. Consequently, normalization without adjudicated cross-family validation can create a false sense of metric harmonization.

The literature's increasing attention to psychometric-style audits points toward a more defensible alternative: treat judges as raters with measurable reliability, bias, and latent-trait interaction effects

rather than as interchangeable black-box scorers (1). In practical benchmark governance, this suggests at least four reforms:

1. **Report judge-conditioned scores rather than only pooled scores.**
   If model A outperforms model B only under one judge family, that should be visible.

2. **Publish uncertainty intervals that include evaluator variance.**
   Confidence intervals based solely on test-set size understate uncertainty.

3. **Distinguish ordinal robustness from absolute-score robustness.**
   A ranking may remain stable while pass/fail thresholds do not, or vice versa.

4. **Freeze evaluator configurations for any claims intended to support external compliance or certification.**
   Otherwise, the metric cannot function as a stable evidentiary instrument.

The economic dimension is also relevant. The RewardBench 2 study found that ensemble scoring can substantially improve accuracy but at approximately 5x cost for the strongest setting, while cheaper model tiers benefited disproportionately from ensembling—e.g., GPT-5.4 mini with (k=8) reached 79.2% at only 1.2x baseline cost (2). This means benchmark comparability is also influenced by resource allocation. Wealthier organizations can buy more reliable evaluations, potentially producing leaderboard positions that are as much about evaluation spend as about model quality. If costlier ensembles become de facto standard without transparent reporting, the benchmark ecosystem risks embedding an unacknowledged "measurement arms race."

There is an additional subtlety: benchmark comparability can fail even when two evaluators yield similar overall scores if they disagree on *why* a model scored as it did. In safety, explanation matters because mitigations are often category-specific. A model that appears acceptable under one judge may in fact have elevated cyber risk but low toxicity risk, while another judge's score obscures the opposite pattern. Therefore, cross-family disagreement can corrupt not only the overall ranking but the allocation of mitigation effort.

A more robust comparability framework would decompose benchmark outcomes into category-level and judge-level tensors, rather than a single table of aggregate scores. For instance:

Model	Judge family	Harmfulness FN rate	Jailbreak detection	Fairness sensitivity	Refusal adequacy	Overall
A	Proprietary family 1	x	x	x	x	x

Model	Judge family	Harmfulness FN rate	Jailbreak detection	Fairness sensitivity	Refusal adequacy	Overall
A	Proprietary family 2	x	x	x	x	x
A	Open-weight family	x	x	x	x	x

Such reporting would allow auditors to see where comparability holds and where it breaks.

For regulatory compliance claims, the immediate implication is that a single benchmark score should not be accepted as evidence of model safety unless the claimant can show evaluator robustness. At minimum, a regulatory submission relying on LLM-as-a-Judge should disclose: the judge family, version, system prompt, aggregation strategy, validation domain, human reference population, and cross-family sensitivity results. Otherwise, the evidence does not meet the standard typically expected of high-stakes measurement instruments

Consequences for AI Safety Leaderboards, Compliance Assertions, and Certification Workflows

Evidentiary status of leaderboard scores: from public ranking signal to weak assurance artifact

While prior sections already established that benchmark scores are wrapper-conditional and that public rankings can become temporally incoherent when judges change, this section addresses a different question: **what kind of evidence a safety leaderboard can legitimately provide once judge instability and judge-targeted attacks are treated as first-order measurement risks rather than background noise**. The central implication is not merely that some scores are "noisy," but that many current leaderboard outputs fail the evidentiary standards normally associated with assurance artifacts used in procurement, regulatory filings, or deployment gates.

A safety leaderboard is often read as if it reports a latent property of the model—e.g., "model (M) is safer than model (N)"—but the newer LaaJ literature increasingly shows that the reported quantity is a composite of model behavior, benchmark design, evaluator family, hidden prompt instructions, aggregation rules, and attack surface. Judge Reliability Harness explicitly frames reliability as a **system configuration** property rather than a model-only property, emphasizing that the judge model, rubric, and prompt templates must all be stress tested and reported alongside scores because

reliability is rarely measured systematically in current practice (1). This is highly consequential for leaderboards that present a single scalar value without exposing the evaluator stack that produced it.

The problem becomes more acute in safety contexts because “good” leaderboard placement is often interpreted not just as comparative performance but as evidence of reduced deployment risk. The methodology discussion in public safety ranking sites already acknowledges an important limitation: many benchmarks primarily test refusal under adversarial pressure, and a model that refuses too broadly could appear strong on jailbreak resistance while being operationally unusable; conversely, refusal scores alone do not capture whether refusals are applied in the right places or whether harmful assistance slips through in subtle forms (2). Once LaaJ instability is added, the issue is no longer only the classic refusal-helpfulness tradeoff. It is that the very instrument used to summarize that tradeoff can itself be non-robust, exploitable, and insufficiently auditable.

The OpenReview benchmark centered on pairwise agreement across 1,994 items from six datasets is especially relevant because it reframes evaluation around standardized agreement rather than relying only on coarse correlation claims (3). This matters for leaderboards because a leaderboard typically compresses rich disagreement structure into a rank order. If judges agree with humans only conditionally—by task type, by prompt format, by model family, or by ambiguity class—then a single public number can obscure the fact that a model’s apparent safety advantage may disappear under a different but equally defensible evaluator configuration. In that setting, the leaderboard ceases to be a stable measurement instrument and instead becomes an artifact of one particular adjudication regime.

The best current benchmark work on judges themselves also undercuts the common assumption that “frontier judge model” is an adequate proxy for evaluator reliability. JudgeBench was created precisely because prior evaluations overly emphasized alignment with human preferences and failed to capture challenging discriminations in knowledge, reasoning, math, and coding; it found that many strong judges perform only slightly above chance on hard pairs, despite otherwise favorable preference-correlation narratives (4; 3). The transfer lesson for safety leaderboards is straightforward: if judges struggle on difficult correctness-grounded discrimination tasks, then safety leaderboards that use those judges to classify nuanced harmfulness, policy compliance, or jailbreak success should not be presumed robust merely because the judge is strong on generic chat preference tasks.

This evidentiary downgrade has several specific consequences for public rankings.

First, **small leaderboard differences are often uninterpretable**. In saturated or variance-heavy benchmark regimes, score differences at the top can be statistically or methodologically meaningless. The broader benchmark critique literature notes that many standard AI evaluations are

saturated at the frontier, so apparent score gaps can be dwarfed by contamination, benchmark gaming, or annotation error (5). In safety leaderboards that add LaaJ-specific variance, this concern is amplified: even if sample-size confidence intervals are narrow, the omitted evaluator-configuration uncertainty may dominate the true uncertainty budget.

Second, **rank order is not equivalent to calibrated risk ordering**. A leaderboard might truthfully state that under benchmark (B), judge prompt (P), rubric (R), and aggregation rule (A), model (M*1) outperformed (M*2). But that does not establish that (M_1) is safer in deployment, safer under realistic attacker adaptation, or safer under independent audit. The gap between lab benchmark scores and deployment performance reported for enterprise agentic systems—37% in one synthesis—illustrates the broader danger of over-reading benchmark results as deployment-ready evidence (5). For safety leaderboards, LaaJ fragility widens that external-validity gap.

Third, **public leaderboards create strategic pressure to optimize for evaluator behavior rather than substantive safety behavior**. This is not merely Goodhart's law in the abstract. When the evaluator is itself a language model vulnerable to position effects, stylistic cues, or prompt-sensitive decision boundaries, developers can often improve score without commensurate improvement in real-world harmfulness reduction. The SoK on LaaJ security explicitly surveys inference-time manipulations, one-token attacks, optimization-based prompt injection, stealthy rubric drift, and backdoor vulnerabilities in judge systems (6). In leaderboard settings, the public and repeated nature of evaluation increases incentives to discover such exploits, while repeated submission opportunities help attackers estimate the judge's decision boundary.

Fourth, **leaderboards can produce a false sense of security for downstream users who are not parties to the original evaluation**. Procurement teams, enterprise integrators, and sectoral auditors often lack access to raw benchmark traces, rejected samples, hidden prompts, or alternative evaluator outputs. They therefore consume the leaderboard as a reputational summary. If that summary is derived from an opaque or unstable LaaJ pipeline, the downstream user inherits not just model risk but also measurement risk, often without visibility into either. This matters because the public presentation layer of a leaderboard—single scores, medals, or category rankings—tends to communicate confidence and comparability even when the underlying evidence is conditional and fragile.

The appropriate interpretation of current safety leaderboards is therefore narrower than common usage suggests. Under the emerging evidence base, most should be treated as **screening tools for hypothesis generation**, not as standalone proof of relative safety, policy compliance, or certification readiness. Table I distinguishes uses that remain defensible from those that become problematic when LaaJ uncertainty is not explicitly quantified.

Use of leaderboard score	Potentially defensible?	Conditions required	Main failure if conditions absent
Early-stage internal triage of model variants	Yes, conditionally	Frozen judge stack, repeated runs, spot human review	Wrong optimization target due to evaluator artifacts
Public comparative ranking across vendors	Weakly	Full evaluator disclosure, re-baselining, uncertainty intervals, appeal process	Misleading rank precision and reputational distortion
Evidence for regulatory compliance	Generally insufficient alone	Independent audits, human validation, traceability, protocol lock	False compliance confidence from unstable metric
Deployment gate for high-risk release	Insufficient alone	Multi-layer evidence, red teaming, domain experts, post-deployment monitoring	Unsafe deployment due to untested evaluator blind spots
Procurement shortlisting	Possibly useful as one signal	Known task relevance, cross-benchmark triangulation, audit access	Vendor choice driven by non-generalizable score
Longitudinal safety trend monitoring	Limited	Frozen or bridged judges, calibration studies, version logs	Apparent improvement/ regression caused by evaluator drift

The emerging governance lesson is that a leaderboard score should be considered an **assurance precursor**, not an assurance endpoint. It may justify deeper examination, but by itself it usually cannot bear the legal, operational, or public-communication weight currently placed on it. This becomes especially important when organizations advertise strong leaderboard positions as if they demonstrated robust security posture. In the presence of LaaJ vulnerabilities, the claim “we score highly on safety leaderboards” can mean anything from genuine broad robustness to successful optimization against one narrow and unstable evaluator stack.

A more defensible public practice would therefore shift from single-number presentation toward **measurement dossiers**: frozen benchmark version, judge family and version, hidden prompt checksum, rubric text, aggregation rule, attack-resilience notes, human-audit sample agreement, and sensitivity analyses across at least one independent judge family. The literature does not yet provide a universal standard, but the direction is implicit in Judge Reliability Harness’s emphasis on

configuration-level reporting and in JudgeBench's insistence that judge quality itself must be benchmarked before being trusted as an evaluator (1; 4).

Failure of compliance-by-benchmark: why regulatory claims inherit judge insecurity

Whereas earlier sections discussed general implications for benchmark governance, this section focuses specifically on **regulatory compliance claims and policy-facing assurance statements**. The key issue is that once LaaJ is used to support claims such as "the system meets organizational safety policy," "the model passes harmful-content thresholds," or "the provider satisfies external risk-management expectations," regulatory assertions inherit the weaknesses of the underlying evaluator. In effect, LaaJ becomes part of the compliance instrumentation, and any instability, exploitability, or opacity in that instrumentation creates compliance risk.

This is especially salient because modern AI governance increasingly depends on documentary evidence: test reports, safety case materials, red-team summaries, incident logs, and benchmark outputs. If an organization relies heavily on LaaJ to generate these artifacts at scale, it may produce voluminous evidence that appears systematic but lacks the measurement robustness regulators normally expect for consequential claims. The issue is not that regulators require perfect determinism; many domains tolerate probabilistic evidence. The problem is that the operating characteristics of LaaJ are often insufficiently characterized, insufficiently disclosed, and insufficiently stable across time.

The public governance discourse already recognizes adjacent requirements. Enterprise guidance associated with safety ranking discussions highlights needs such as **audit logging**, **data handling transparency**, **customizable safety policies**, and **incident response processes**, and notes that the EU AI Act will require logging capabilities for high-risk systems by August 2026 (2). These requirements imply that compliance evidence must be inspectable and attributable. Yet a typical LaaJ pipeline often stores only final labels or aggregate scores, not the full evaluation context: exact model endpoint, prompt wrapper, rubric version, candidate serialization, reasoning traces if any, retries, temperature, truncation events, parser failures, or judge abstentions. A compliance claim built on such incomplete traces is difficult to audit and difficult to contest.

The new concern raised by the LaaJ security literature is that **compliance failure can arise without any change in the evaluated model's real-world behavior**. Several attack families surveyed in the SoK manipulate the judge rather than the target model: prompt injection into the evaluation context, optimization-based attacks on the judge prompt, one-token perturbations, emoji-mediated attacks, rubric attacks that induce preference drift, and even backdoors implanted in judge behavior (6). If

safety or policy compliance is assessed through such a judge, then the organization may appear compliant or non-compliant based on evaluator manipulation rather than genuine system properties.

This creates at least four regulatory problems.

1) Compliance claims become protocol-relative rather than system-relative.
A provider may truthfully state that it passed benchmark (B) under protocol (P). But if modest changes to rubric wording, candidate presentation, or judge family alter pass/fail status, then the compliance claim is tied to the protocol rather than to the model. This is problematic because regulators and customers often interpret benchmark passage as evidence about the system itself. Judge Reliability Harness directly challenges this by proposing systematic stress testing of judge configuration and by noting that such reliability characteristics are rarely reported alongside evaluation results (1).

2) There is increased risk of selective disclosure and compliance cherry-picking.
Where multiple defensible judge configurations exist, an organization can choose the one producing the most favorable result. This need not involve overt bad faith; teams naturally gravitate toward protocols they consider "reasonable." But in the absence of mandatory evaluator disclosure and sensitivity analysis, the public or regulator sees only the selected score. This creates an unobserved researcher-degrees-of-freedom problem in compliance reporting. The danger is particularly high when benchmark definitions are not fully standardized or when hidden system prompts are proprietary.

3) Policy thresholds can become unstable at the decision boundary.
Suppose a policy says a model may be deployed if harmful-content violation rate stays below (x\%). If judge perturbations move the measured rate by several points—or even alter which outputs count as violations—the pass/fail decision becomes non-robust. In such cases, the legal and operational significance attached to the threshold far exceeds the measurement precision of the instrument. The 2025 paper on evaluating metrics for safety with LLM-as-judges argues that weighted baskets of metrics and concordance-triggered human review may be necessary precisely because deterministic evaluation is unavailable and single-metric reliance is too risky (7).

4) Audit defense becomes weak when adverse findings are challenged.
A compliance process must not only produce judgments but withstand dispute. If a provider is accused of misrepresenting risk or if an audited party contests an adverse safety finding, the evaluator must be explainable and reproducible enough to support adjudication. Opaque LaaJ pipelines fail here because they rarely support exact reruns, stable versioning, or clear attribution of why a sample was classified as harmful, safe, or policy-violating. In legal or quasi-legal settings, this weakens the defensibility of the benchmark evidence.

These concerns can be made concrete by mapping LaaJ failure modes onto compliance control failures, as shown in Table II.

LaaJ weakness	Compliance artifact affected	Typical downstream claim at risk	Why claim becomes fragile
Hidden prompt/ rubric changes	Test report, internal control evidence	"Model passed standardized safety evaluation"	Standardization is not demonstrable if evaluator definition is opaque
Judge-family substitution	Cross-period compliance comparisons	"Safety improved quarter-over-quarter"	Apparent improvement may reflect evaluator change
Candidate-side judge manipulation	Red-team and harmfulness assessments	"No successful jailbreaks observed"	Attack may succeed in deployment while evading judge detection
Shared-model blind spots	Internal policy conformance tests	"Evaluator catches prohibited assistance consistently"	Judge may miss failure modes similar to its own family's weaknesses
Preference leakage/ contamination	Third-party validation reports	"Independent evaluation confirms provider's policy alignment"	Evaluation may be contaminated by overlap with training or known preferences
Missing logs and version metadata	Audit trail	"Results are reproducible and reviewable"	Rerun and challenge procedures cannot be executed faithfully

The contamination angle deserves separate emphasis. The paper on **Preference Leakage** argues that LLM-as-a-judge systems suffer a contamination problem in which evaluator preferences can be leaked or reflected in ways that distort apparent validation, and it situates this within a larger literature on egocentric bias and other vulnerabilities of LLM judges (8). For compliance use, contamination is more than a methodological nuisance. If a provider's judged outputs are evaluated by a model family with overlapping preferences, training exposure, or shared response style, the resulting "independent validation" may not be substantively independent. This is analogous to a compliance test partially keyed to the design assumptions of the product being tested.

A related issue concerns **regulatory over-translation**: importing a benchmark score into legal or policy language that implies more than the benchmark justifies. Statements such as “compliant with safety requirements,” “enterprise-ready,” or “meets high-risk system obligations” are often broader than what the underlying benchmark measured. If the benchmark itself used LaaJ with unreported uncertainty, the semantic gap widens further. The formal problem is not merely lack of construct validity, but the conversion of a conditional measurement into a categorical claim with institutional consequences.

The challenge is intensified by fast-moving model and API ecosystems. Even if a provider initially validated a LaaJ pipeline against humans, subsequent upstream changes can alter evaluator behavior without immediate notice. The SoK literature emphasizes that judge systems can be targets of both training-time and inference-time threats, while broader benchmark critiques note that static validation does not guarantee ongoing reliability (6; 5). Thus, a compliance claim is not just vulnerable at time of filing; it can silently degrade afterward if the evaluator stack drifts.

For this reason, compliance claims built on LaaJ should be treated as requiring **continuous validity maintenance**, not one-time validation. At minimum, that implies:

1. frozen evaluator specifications for any reported claim period;
2. full trace logging sufficient for rerun or third-party inspection;
3. sensitivity analysis across multiple judge families or prompting variants;
4. human review of edge cases and threshold-near cases;
5. explicit statement of what construct was measured and what was not.

Without these controls, the most likely failure mode is not obvious fraud but **false security**: an organization genuinely believes it has strong documentary evidence of safety compliance when in fact the evidence is brittle, partially evaluator-specific, and contestable under replication.

This has immediate implications for how providers should present claims to customers and regulators. Table III contrasts weak and stronger formulations.

Claim formulation	**Assessment**	**Reason**
“Our model is compliant with safety requirements because it ranked top on benchmark X.”	Weak / misleading	Converts conditional benchmark success into broad compliance claim
“Under benchmark X, using frozen judge version Y and rubric Z, the model achieved score S;	Stronger	Binds claim to protocol and discloses part of validity basis

Claim formulation	Assessment	Reason
independent human audit on sampled edge cases showed agreement A.”		
“The model has been certified safe by automated evaluation.”	Weak	Treats automation as sufficient and ignores residual uncertainty
“Automated evaluation was used as one screening layer within a broader assurance case including human review, red teaming, and monitoring.”	Stronger	Accurately positions LaaJ as partial evidence
“Quarterly improvements in safety score demonstrate reduced real-world risk.”	Weak	Assumes temporal comparability and external validity not yet established
“Quarterly score changes were observed under a fixed evaluation stack; independent checks are ongoing to assess whether the changes correspond to deployment-relevant risk reduction.”	Stronger	Separates observed metric change from validated safety improvement

The regulatory bottom line is not that LaaJ is unusable. Rather, it is that **compliance-by-benchmark is unsafe when benchmark evidence is treated as self-authenticating**. In regulated or quasi-regulated settings, LaaJ should be handled as a fallible measurement subsystem whose own security, stability, and traceability must be demonstrated before its outputs can support consequential claims.

Certification and release gating under non-replicable evaluation: chain-of-custody, replayability, and dispute resolution

This section differs from prior discussions of deployment certification in one important respect. Earlier material argued that one must specify the judge, version, wrapper, and execution environment. Here the focus is on the **operational mechanics** of certification: how non-replicable LaaJ outputs break chain-of-custody, undermine replayability, and weaken dispute resolution when safety approval or release gating depends on automated judgments.

Certification processes—whether internal release sign-off, customer-facing attestation, third-party audit, or formal conformity assessment—require more than a score. They require a **verifiable path** from test design to final decision. In mature assurance domains, that path includes version control,

provenance, environmental reproducibility, test rerun procedures, evidence retention, and procedures for handling contested findings. LaaJ pipelines often underperform on all of these dimensions because the evaluator itself is opaque, vendor-hosted, updated asynchronously, and embedded in a prompt-and-parser stack that is not routinely treated as certifiable infrastructure.

The benchmark literature suggests a large underlying problem: many evaluation ecosystems still treat the judge as an interchangeable utility rather than as a measurement instrument with its own attack surface and calibration burden. Judge Reliability Harness specifically responds to this gap by proposing systematic reliability stress testing of the full judge system configuration (1). But certification workflows demand still more: not merely ex ante stress testing, but evidence retention and replayability that allow a later auditor to reconstruct exactly how a pass/fail decision was produced.

This requirement becomes acute in safety-critical release gating. Consider a deployment gate that allows launch only if the model's measured harmfulness rate remains below a threshold across several categories. If a later incident occurs—say, the model provides dangerous assistance in a domain the certification process allegedly screened out—investigators will ask whether the failure was due to model drift, benchmark insufficiency, evaluator manipulation, or simple misclassification. Without complete LaaJ traceability, these questions become unanswerable. The organization may know the aggregate score but not the exact sequence of prompts, judge model build, intermediate outputs, or parsing events that produced it.

The problem can be decomposed into five auditability failures.

A. Incomplete chain-of-custody for evaluation artifacts

A certifiable evaluation must preserve provenance for:

- benchmark dataset version,
- prompt templates,
- rubric text,
- serialization order of candidates,
- judge model family and version,
- decoding parameters,
- retries/timeouts,
- parsing and normalization code,
- aggregation rules,
- human overrides or abstentions.

In many production settings, only a fraction of this information is retained. Public benchmark dashboards rarely publish it, and even internal systems may store only final scores. Yet each of these components can affect outcome. The SoK literature shows that attacks can target prompt structure, rubric content, or specific token-level vulnerabilities (6). If those artifacts are not preserved, subsequent audit cannot establish whether a certification outcome reflects substantive model behavior or an exploitable evaluation condition.

B. Lack of replayable environments

Even when prompts are logged, exact replay may fail because the judge is served through a changing API endpoint. Frontier model providers routinely update safety behavior, hidden prompts, and inference stacks. Unless the certification process uses a frozen, reproducible evaluator environment, later reruns can yield different answers even with identical visible inputs. That means certification evidence is not self-verifying over time. A release decision approved in March may be impossible to reconstruct in September.

This undermines both internal governance and external assurance. Internally, safety teams cannot tell whether a changed score reflects improved target-model behavior or changed judge behavior. Externally, customers and auditors cannot independently confirm what was claimed at the time of certification. The issue is not unique to LaaJ, but LaaJ makes it worse because the evaluator is a generative model with higher behavioral variability than conventional software metrics.

C. Weak dispute resolution procedures

A robust certification regime must handle contested cases: e.g., the model provider disputes an adverse audit finding, or a customer claims the certifying party missed a harmful behavior. With human review panels, dispute resolution can proceed by re-reading evidence, consulting experts, and documenting reasoning. With LaaJ-only certification, disputes often reduce to “the model said so,” especially if the judge’s reasoning is itself unstable or nontransparent.

The agreement-centric evaluation work on 1,994 items across six datasets is relevant here because it suggests the importance of measuring not just average correlation but patterns of disagreement among judges and humans (3). In certification, disagreement analysis is essential: contested cases are precisely where confidence should be lowest and review most intensive. Yet standard LaaJ pipelines seldom expose disagreement profiles, uncertainty flags, or adjudication pathways.

D. Threshold fragility in categorical release gates

Certification often operationalizes continuous scores into discrete decisions: approved/not approved, compliant/non-compliant, deploy/block. This conversion magnifies any evaluator instability near

the boundary. A one-point score shift from a prompt change, parser adjustment, or judge update can flip a decision with large organizational and legal consequences. The 2025 work on metric design for safety explicitly recommends weighted metric baskets and confidence thresholds triggering human review when evaluator concordance is low (7). That recommendation can be read as a direct response to threshold fragility: do not let a single unstable judge own the final gate.

E. Silent contamination of the certifier's independence

Certification is often trusted because the certifier is presumed independent from the system under evaluation. But if both evaluated model and judge share training data, architecture family, provider infrastructure, or policy style, that independence can be compromised in subtle ways. The **Preference Leakage** literature frames one aspect of this as contamination of the judging process itself (8). In certification settings, such contamination weakens the evidentiary force of "independent automated assessment," especially when no external human or cross-family validation is performed.

These auditability failures have direct consequences for deployment governance. Table IV presents a certification-oriented risk matrix.

Certification function	What LaaJ is often used for	Auditability requirement	Typical gap
Pre-release safety gate	Classify harmful outputs, estimate violation rate	Exact replay of failing and passing cases	Judge/version/API drift prevents replay
Supplier attestation	Demonstrate policy conformance to customers	Traceable protocol and sample-level evidence	Only aggregate scores disclosed
Third-party audit support	Scale evaluation over many prompts	Independent re-execution and challenge procedures	Proprietary prompts or judge stack block re-audit
Post-incident investigation	Reconstruct why unsafe behavior was missed	Full chain-of-custody and classification rationale	Missing logs, hidden wrappers, non-deterministic outputs
Ongoing surveillance	Track degradation after deployment	Stable baseline and change-control records	

Certification function	What LaaJ is often used for	Auditability requirement	Typical gap
			Upstream evaluator changes confound monitoring

The result is a mismatch between the **institutional role** certification is expected to play and the **epistemic quality** of the evidence often supplied. Certification language implies repeatability, challengeability, and documented process control. LaaJ-generated evidence often provides scale but not those assurance properties.

This should change how organizations design safety sign-off procedures. A certifiable LaaJ pipeline should be treated more like a validated laboratory assay or a regulated measurement device. That does not mean it must be perfectly deterministic, but it should satisfy at least the following:

- frozen evaluator release or self-hosted version pinning for certification cycles;
- immutable logging of full prompts, outputs, metadata, and parsing events;
- cryptographic hashes for benchmark corpora, rubric files, and prompt templates;
- pre-specified abstention and escalation rules;
- replay harnesses that can regenerate prior judgments or explain why not;
- recorded human adjudication on contested or high-severity items;
- change-control procedures for any evaluator update affecting certified claims.

An underappreciated point is that **auditability is itself a safety control**, not merely a compliance convenience. If LaaJ evaluations cannot be replayed or contested, then organizations lose an essential learning mechanism after incidents. They cannot determine whether the benchmark missed a failure mode because the target model improved at evasion, because the judge was manipulated, or because the prompt/rubric stack drifted. This hampers corrective action and allows false confidence to persist.

The benchmark ecosystem outside LaaJ provides cautionary context. Broader evaluations have reported high annotation error rates and benchmark gaming, while emphasizing the need for expert review in production-grade validation (5). In LaaJ-based certification, the danger is stronger because the errors can be systematic yet invisible: a whole class of harmful outputs may pass unnoticed if the judge shares the same stylistic or conceptual blind spot as the evaluated model family.

For high-stakes deployment, then, certification should not be a single terminal decision produced by LaaJ. It should be a **documented adjudication process** in which LaaJ contributes scalable

evidence, but final authority depends on replayable traces, uncertainty-aware thresholding, and human review where the cost of error is high. Without that shift, deployment certificates and release approvals risk becoming polished artifacts of non-replicable automation rather than durable evidence of safety.

Auditability deficits as a governance vulnerability: logging, explainability, and post-market accountability

This section is distinct from the previous one in that it moves beyond certification-time replayability to the broader governance question of **how auditability deficits impair monitoring, incident response, and accountability after deployment**. The point is not simply that missing logs make certification weaker; it is that poor LaaJ auditability can obstruct the entire lifecycle of safety governance, including external review, root-cause analysis, corrective action, and public trust.

The emerging AI governance environment increasingly expects organizations to maintain records of model behavior, testing procedures, safety interventions, and incident handling. Public-facing enterprise discussions already flag audit logging and retrievability of safety filter activations as important operational capabilities, with explicit relevance to forthcoming legal requirements such as those under the EU AI Act for high-risk systems (2). If LaaJ becomes a central component of safety assessment, then the judge pipeline itself enters the audit perimeter. Yet many organizations still log LaaJ outputs as if they were ordinary analytic metrics rather than security-relevant adjudications.

This is dangerous for three reasons.

First, **LLM judges are not passive instruments**. Unlike fixed statistical metrics, they interpret instructions, respond to prompt context, and can be manipulated by the very artifacts they evaluate. Consequently, audit logs must capture not only final labels but the full semantic context in which the judgment occurred. The SoK explicitly frames LaaJ as exposed to both training-time and inference-time threat vectors, and surveys prompt injection, token-level exploits, backdoors, and rubric attacks as live security concerns (6). A final “safe/unsafe” label without context is therefore forensically inadequate.

Second, **the absence of detailed logs masks evaluator-side failure modes that would otherwise be diagnosable**. Suppose a harmful output is wrongly scored as benign. Possible causes include: serialization error, prompt truncation, accidental leakage of benchmark metadata into the judge context, parser bug, evaluator refusal, adversarial prompt injection, or latent judge bias. If only the aggregate score is stored, none of these hypotheses can be tested. Post-market governance then degenerates into speculation.

Third, **lack of auditability undermines accountability allocation**. In multi-party settings—model developer, benchmark operator, third-party auditor, customer, regulator—responsibility for a flawed safety assessment can only be assigned if the evidence chain is inspectable. Otherwise, each party can plausibly attribute the failure elsewhere: the developer blames the benchmark, the benchmark operator blames the upstream judge API, the auditor blames missing vendor disclosure, and the customer is left with no clear remediation path.

To make this concrete, Table V lists the minimum logging fields that a governance-capable LaaJ system should preserve, together with the accountability purpose each serves.

Logged field	**Why it matters for governance**	**Failure if missing**
Judge model identifier and version	Establishes which evaluator made the judgment	No attribution; rerun impossible
Full system/user prompts and rubric	Allows scrutiny of instruction context	Hidden prompt effects cannot be investigated
Candidate order and serialization format	Necessary to detect order and formatting artifacts	Position-sensitive errors remain invisible
Raw judge output before parsing	Preserves ambiguous or hedged responses	Parsing-induced label errors cannot be audited
Parsing rules and code version	Shows how free text became discrete score	Silent parser changes can alter results
Decoding parameters / retries	Captures non-determinism and fallback behavior	Result reproducibility overstated
Timestamp and endpoint metadata	Supports drift analysis and change tracking	API or infrastructure changes go unnoticed
Human override/ escalation flags	Distinguishes automated from adjudicated outcomes	False impression of full automation or full review
Confidence/disagreement measures	Supports risk-based escalation	Threshold decisions appear cleaner than they are

Logged field	Why it matters for governance	Failure if missing
Attack or anomaly indicators	Helps detect manipulation attempts	Judge-side compromise may masquerade as clean result

In practice, most public safety leaderboards expose almost none of this information. Even internal benchmark systems often log only a subset. This creates a structural governance problem: the most visible safety claims may be supported by the least inspectable evidence.

The issue of **explainability** is similarly misunderstood. It is tempting to ask an LLM judge to "explain its decision" and treat that explanation as an audit artifact. But this can be misleading. Explanations generated after classification may not faithfully reflect the causal basis of the judgment, and they may vary across runs. Still, some explanation is better than none if handled properly. The correct role of judge-generated reasoning in governance is not as conclusive proof but as one signal to be archived, compared across models, and tested against decision consistency. Explanation should support audit, not replace it.

The agreement-centric benchmark work offers one promising perspective here. By mixing LLM judges with human raters and computing pairwise agreement across 1,994 items from six datasets, it creates a framework in which disagreement itself becomes observable and measurable (3). For governance, this is valuable because unexplained disagreement is often a stronger signal of evaluation risk than high mean performance is a signal of reliability. A well-audited LaaJ pipeline should therefore record not only what final label was assigned but whether alternative judges or humans disagreed, and on which classes of items.

Auditability deficits also interact badly with **post-market monitoring**, an area likely to become more important as deployed foundation models are updated continuously. If an organization uses LaaJ to monitor whether a deployed system is becoming more or less safe over time, weak logging prevents it from separating real safety change from evaluator drift. This is more than a technical nuisance. In governance terms, it means trend reports may be uninterpretable, incident thresholds may be triggered incorrectly, and corrective action may target the wrong subsystem.

There is also a public-trust dimension. Safety benchmarks and audits increasingly function as trust signals to users, enterprise buyers, and policymakers. If later analysis reveals that a headline safety claim cannot be reconstructed because logs were incomplete or the judge endpoint changed without record, confidence in the broader assurance ecosystem may erode. In that sense, LaaJ auditability failures create not only local risk but **ecosystem-level legitimacy risk**.

A more governance-appropriate LaaJ design would incorporate the following principles:

1. **Forensic completeness**: log enough context to diagnose misjudgments, not just enough to reproduce aggregate metrics.
2. **Change transparency**: every evaluator update should trigger version increments, compatibility notes, and where relevant re-baselining.
3. **Disagreement preservation**: store minority or alternative judgments rather than collapsing immediately to a single score.
4. **Escalation traceability**: record when humans reviewed cases, what they saw, and how final adjudication was reached.
5. **Retention discipline**: preserve logs long enough to support regulatory inquiry, incident investigation, and procurement disputes.

The literature does not yet provide a mature standardized audit schema for LaaJ, but the need is strongly implied by existing evidence. Judge Reliability Harness identifies the lack of systematic reporting of judge-system reliability as a core gap (1); the SoK catalogues attack surfaces that make sparse logging unsafe (6); broader benchmark critiques emphasize that production-ready evaluation requires layered approaches with human review (5). Together, these imply that governance claims unsupported by rich audit trails should be considered weak even when the benchmark numbers look favorable.

The practical implication for organizations is not merely "log more." It is to treat LaaJ judgments as **regulated evidence objects**: each judgment should carry metadata, provenance, and contestability features proportionate to the decision it informs. If the judgment can affect release, certification, incident response, or legal representation, then sparse logging is not operational efficiency; it is a governance defect.

Assurance-oriented mitigations: ensemble adjudication, judge benchmarking, and human escalation as controls against false security

This section complements earlier mitigation discussion by focusing specifically on **which countermeasures are most relevant when the target problem is false security in leaderboards, compliance claims, and certification**, rather than judge instability in the abstract. The key question is not whether mitigations improve average benchmark accuracy, but whether they make evaluation outputs more suitable for high-stakes assurance use.

Three mitigation families dominate the recent discussion:
1.

Mitigation Strategies and Assurance Frameworks for Security-Robust LLM-as-a-Judge in AI Safety Assessment

From “use multiple judges” to defensible adjudication architectures

While earlier sections already noted that ensembles can reduce variance and that high-stakes cases should escalate to humans, the key issue here is narrower and more operational: **which ensemble designs actually improve security and assurance properties in AI safety evaluation, and under what failure correlations do they stop helping?** The contemporary literature suggests that naïve plurality voting across similar judges is inadequate; what matters is structured heterogeneity, calibrated aggregation, adversarial coverage, and explicit uncertainty reporting.

A first lesson from the recent robustness literature is that judge failures are often **systematic rather than independent**. In the SoK on security in LLM-as-a-Judge, the threat landscape is organized around both attacks *against* the judge and cases where the judge acts as a vulnerable instrument inside evaluation pipelines. This framing matters because it implies that an ensemble is only as strong as the diversity of its failure modes: if all members share susceptibility to prompt injection, stylistic deception, or family-specific priors, aggregation can merely stabilize the wrong answer rather than recover the right one (1). The same SoK notes that mitigation proposals in the literature cluster around auditing, misuse monitoring, and explainability rather than any single universally effective defense, underscoring that “ensemble” is a design space, not a turnkey control (1).

The strongest empirical caution comes from robustness benchmark work summarized in the SoK’s HTML version. The RobustJudge evaluation reports that prompt-template selection alone can move robustness by **up to 40 percentage points**, and that composite attacks remain successful even against commercially deployed systems. The “Combined Attack,” which stacks manipulations such as context ignoring, escape characters, and fake completions, reaches extreme success rates on some judges; even GPT-4o-level judges are reported to show attack success rates around **70%** under strong methods such as PAIR and composite attacks in certain evaluation settings (2). This implies that an ensemble drawn from similarly prompted general-purpose models may exhibit *shared prompt-template fragility*. Therefore, assurance-oriented ensemble design must vary not just model identity but also prompt wrappers, evaluation formats, and attack-exposure conditions.

A useful way to structure the design problem is to decompose an LLM judge panel into five dimensions, shown in Table I.

Dimension	Weak implementation	Stronger implementation	Security rationale
Model diversity	Same provider, same family, same alignment style	Cross-provider and cross-family panel	Reduces shared blind spots and inherited preference leakage
Prompt diversity	One fixed system prompt	Multiple independently authored prompts / wrappers	Reduces template-specific exploitation
Task framing	Single pairwise preference task	Mixed pairwise, rubric-based, error-tagging, and abstention-enabled tasks	Makes attacks harder to optimize against a single scoring channel
Aggregation rule	Simple majority vote	Reliability-weighted fusion with disagreement flags	Avoids over-trusting weak or correlated judges
Escalation	None	Human review on contested, high-impact, or attack-suspected cases	Handles adversarial and distribution-shift failures

Table I. Assurance-relevant dimensions of judge ensemble design.

The literature on communication-system evaluation provides more explicit mitigation language than much of the benchmarking literature. A recent study on mitigating LLM-as-a-judge bias recommends "a panel of diverse judges" and explicitly argues that multiple LLMs from different providers or training backgrounds can dilute individual biases; it also stresses human oversight in sensitive contexts (3). Although that paper is domain-specific and not itself a security benchmark, its contribution is conceptual: **ensemble value comes from disagreement structure**, not merely count. For AI safety evaluation, the practical extension is that panel construction should be threat-model-driven. If the evaluation target is jailbreak detection, one should include judges optimized for harmful-content identification, judges trained to verify semantic entailment of policy violations, and perhaps symbolic or retrieval-backed checkers for cited policy clauses. If the target is refusal adequacy, at least one judge should operate with a rubric that separates refusal tone from refusal sufficiency, to reduce style-driven false positives.

A second lesson is that aggregation should not collapse disagreement too early. Standard majority voting converts a rich diagnostic signal into a brittle verdict. Recent "meta-judging" descriptions emphasize weighted averaging, voting panels, and multi-stage filtering; one synopsis reports up to **15 percentage-point precision improvements** from architectures that critically score and filter raw

judge outputs rather than merely averaging them (4). Because this source is a secondary synthesis rather than a primary peer-reviewed paper, the exact magnitude should be treated cautiously; still, it points to an increasingly important distinction between **first-order judges** and **second-order adjudicators**. In assurance terms, a second-order component can inspect whether the judges agree for the same reasons, whether they cite policy-relevant evidence, and whether the disagreement pattern itself indicates attack or contamination.

That design principle aligns with psychometric views of LLM evaluation. The systematic review on LLM psychometrics notes reliability dimensions such as internal consistency, parallel forms, inter-rater reliability, option-position robustness, and adversarial robustness (5). Translated into adjudication architecture, this means a trustworthy panel should be treated less like a one-shot prediction service and more like a **measurement instrument** requiring parallel forms and consistency checks. For instance, one can run the same safety item under paraphrased rubrics, reversed candidate order, and multiple evaluation temperatures; then aggregate not just the scores but the observed sensitivity. A model that "passes" only when one narrow prompt template is used should be treated as provisionally passing at best.

This directly connects to hidden measurement error. The 2026 paper on hidden measurement error argues that changing the prompt, judge model, or temperature can shift results enough to **flip rankings and reverse conclusions**, and that ordinary confidence intervals severely under-cover because they ignore this source of variance. The paper's core governance implication is that benchmark builders must decompose uncertainty into components due to sampling and due to design choices, because the latter creates exploitable surface for gaming (6). For ensemble design, this means aggregation rules should output at least three things: a central score, a **between-judge variance estimate**, and a **design-sensitivity interval**. Without those, a panel can give a false sense of precision while still being easy to game.

One implication is that there are at least four distinct ensemble purposes in AI safety evaluation, summarized in Table II.

Ensemble purpose	**Primary objective**	**Typical mechanism**	**Failure if misused**
Noise reduction	Reduce stochastic score variance	Repeated runs, averaging	Masks systematic bias
Bias balancing	Offset idiosyncratic stylistic or ideological preferences	Cross-family and cross-provider judges	Fails if biases are correlated

Ensemble purpose	Primary objective	Typical mechanism	Failure if misused
Attack resistance	Increase cost of adversarial manipulation	Diverse wrappers, hidden prompts, challenge rounds	Security through obscurity if not stress-tested
Assurance evidence	Produce auditable, uncertainty-aware decisions	Weighted fusion + logs + human escalation	Becomes ceremonial if uncertainty is not reported

Table II. Distinct functions of judge ensembles in safety-critical evaluation.

A stronger ensemble protocol for safety certification would therefore include the following procedural steps:

1. **Panel construction by residual-error diversification.** Choose judges whose errors are empirically shown to differ on adversarial and benign perturbation sets, rather than selecting models solely by average agreement with humans.
2. **Parallel-form scoring.** Evaluate each item under alternative rubric formulations, candidate orders, and formatting controls.
3. **Open-set disagreement handling.** Permit abstention or "insufficient confidence" outputs, rather than forcing a scalar score on every item.
4. **Evidence-aware aggregation.** Weight judges by domain calibration and robustness statistics, not simply by provider reputation.
5. **Escalation triggers.** Route cases with high disagreement, attack signatures, or threshold-near outcomes to human review.

This is also the point at which **human-AI hybrid adjudication** becomes more than a fallback. In recent work on patch evaluation, the proposed direction is explicitly "towards a human-in-the-loop framework for reliable patch evaluation using an LLM-as-a-judge" (7). Even though the report excerpt provides only a citation and not detailed quantitative results, the framing is important: in domains where a wrong judgment has operational consequences, the correct use of LLM judges is often triage, standardization, and pre-screening rather than autonomous final adjudication.

The practical ensemble question is thus not "How many judges should we use?" but "How do we make disagreement informative?" A robust architecture should treat disagreement as a first-class output. For example:

- **Low disagreement + low sensitivity across perturbations** supports a stronger evidentiary claim.
- **Low disagreement + high shared attack susceptibility** supports only a weak claim; consensus may be illusory.
- **High disagreement concentrated near decision thresholds** should trigger human arbitration.
- **High disagreement concentrated on one judge family** may reveal contamination or preference leakage in that family.

A panel that exposes these structures can support assurance better than a single high-performing judge, even if its raw average correlation with human labels is only modestly better.

The contamination literature reinforces this point. The "Preference Leakage" paper argues that when generator and evaluator models are closely related, evaluations can be systematically biased and that the problem is pervasive in popular benchmarks such as AlpacaEval 2.0 and Arena-Hard because advanced LLMs like GPT-4 are used both for synthesis and evaluation (8; 9). An ensemble that includes related evaluators without accounting for this dependency can produce *overconfident but contaminated* agreement. A better design would include at least one evaluator from a demonstrably independent family and a contamination audit that measures whether model rankings materially change when related judges are removed.

For AI safety uses, this matters because many safety benchmarks are thresholded: a model either passes a safety gate, qualifies for a deployment setting, or supports a compliance assertion. Under such conditions, a small ensemble-induced score shift can change a binary governance decision. The measurement-error literature gives a sharp illustration of the general instability problem: semantics-preserving perturbations can account for up to **half of performance variance**, answer-ordering changes can shift MMLU rankings by **up to 8 positions**, a null model can obtain an **86.5% win rate** on AlpacaEval 2.0, and removing **fewer than 10 preferences out of roughly 40,000** on Chatbot Arena can flip the top-ranked model (6). Although these examples are not all safety-specific, they are directly relevant to safety adjudication because they show how much outcome space remains under evaluator and benchmark design control. Ensembles should therefore be judged not only by average agreement but by their ability to **shrink exploitability**.

This leads to a more assurance-relevant benchmark for ensemble quality: not "Does the panel agree with human judgments on average?" but "How much does the panel reduce ranking reversals,

threshold flips, attack success, and contamination sensitivity relative to a single judge?" A mature safety pipeline would report these reductions explicitly. For example, an evaluation card for a judge panel could state:

- attack success under benchmarked manipulations,
- sensitivity to candidate order,
- score variance under rubric paraphrases,
- cross-family rank stability,
- contamination sensitivity when related judges are excluded,
- human-overturn rate on escalated cases.

Such reporting is still uncommon, but the literature now provides enough conceptual and empirical scaffolding that its absence should increasingly be viewed as an assurance deficit rather than mere immaturity.

Meta-evaluation as infrastructure: benchmarking the judge before trusting the benchmark

Where prior sections discussed the unreliability of benchmark scores under perturbation, this section focuses on the emerging response: **meta-evaluation benchmarks that evaluate the evaluator itself**. This is a distinct layer of control. Instead of asking only whether a target model appears safe, a meta-evaluation framework asks whether the judging apparatus is calibrated, robust, contamination-resistant, and fit for the specific adjudicative role assigned to it.

The need for this layer is increasingly explicit in the literature. The SoK on security in LLM-as-a-Judge emphasizes "the lack of standardized evaluation methodologies" and "the limited availability of robust benchmarks for assessing the security and reliability of LaaJ frameworks" as major research gaps (1). In other words, many benchmark ecosystems still validate model outputs with instruments that have not themselves been subjected to comparably rigorous benchmark validation. For AI safety uses—where benchmark outputs may inform release gates, procurement decisions, or policy claims—this inversion is untenable.

A credible meta-evaluation benchmark for LLM judges should test at least six dimensions:

Dimension of judge quality	Core question	Why it matters for safety assurance
Criterion validity	Does the judge track the intended latent construct?	Avoids certifying style or conformity instead of safety

Dimension of judge quality	Core question	Why it matters for safety assurance
Robustness	Do judgments survive benign and adversarial perturbations?	Prevents benchmark gaming and manipulation
Calibration	Does confidence correspond to correctness?	Supports escalation and abstention policies
Contamination resistance	Is the judge independent of the evaluated model/data pipeline?	Reduces preference leakage and family favoritism
Procedural stability	Are outcomes stable across prompts, templates, and versions?	Needed for replayability and auditability
Interpretability / evidence use	Can the system justify decisions in policy-relevant terms?	Supports audits, disputes, and remediation

Table III. Core dimensions for judge meta-evaluation in high-stakes AI safety settings.

Current work touches these dimensions unevenly. RobustJudge, as summarized in the SoK HTML, provides one of the most concrete robustness-oriented exemplars by benchmarking **15 attack methods** and **7 defense strategies** across **12 models**, spanning closed-source, open-source, fine-tuned, and reasoning-specialized judges (2). This breadth is notable because many older studies tested one or two attack styles on one or two judges, leaving open the possibility that apparent robustness was benchmark-specific. A meta-evaluation suite of this broader kind is valuable not just for attack cataloging but for **comparing defense transferability**: a defense that works on one judge family but fails on another is weak evidence for certification use.

The robustness findings also alter what a meta-benchmark should report. If prompt-template choice alone can change robustness by up to **40 percentage points**, then reporting a single attack success rate for a judge without specifying the prompt wrapper is insufficient (2). Meta-evaluation protocols should therefore benchmark **configuration bundles**, not abstract model names. A "judge" in assurance practice is not merely "GPT-4o" or "JudgeLM-13B"; it is model version + system prompt + rubric + decoding settings + input preprocessor + aggregation rule + logging policy. This bundle-level view is consistent with the configuration-aware perspective emerging elsewhere in the literature on reliability harnesses and hidden measurement error, though the latter is especially explicit in showing that design-choice variance is substantively different from sample-size variance (6).

A second frontier is **benchmarking contamination and relatedness**, not only adversarial manipulation. Preference leakage is especially important here because it creates a failure mode that can look like strong evaluator performance. The contamination paper argues that relatedness between data generators and evaluators can systematically bias evaluations and that the issue is widespread in leading LLM-as-a-judge benchmarks and studies (9; 8). For meta-evaluation, this means one should include *holdout judge families* and *cross-family swap tests*: if a benchmark ranking changes sharply when a closely related evaluator is replaced by a less related one, the benchmark should be flagged as contamination-sensitive.

This kind of meta-evaluation is not just a technical nicety. It has direct implications for safety claims. Suppose a model scores highly on a harmful-content refusal benchmark judged by an evaluator from the same family as the training data generator, and that evaluator's ranking is later found to diverge materially from an unrelated family under paraphrased rubrics. In that case, the original score is not merely noisy; it may be measuring alignment with a family-specific preference manifold rather than generalizable safety performance. A meta-benchmark that can reveal such divergence before the score is operationalized is therefore part of the assurance boundary.

Meta-evaluation must also include **stress tests against benchmark gaming**. The hidden measurement error paper gives multiple examples of systems achieving strong apparent performance through exploitability rather than genuine capability, including the striking result that a fixed-response "null model" can achieve an **86.5% win rate** on AlpacaEval 2.0 (6). This indicates that judge-facing artifacts—verbosity, genericity, positional quirks, or latent benchmark priors—can be optimized independently of the target competence. A meta-evaluation benchmark should therefore include *adversarially optimized low-quality submissions* whose purpose is to fool the judge. If a judge cannot separate these from truly safe or useful outputs, its use in safety certification is questionable.

The W&B overview, though not a primary academic source, captures this operationally: formatting tricks, keyword stuffing, and verbose padding can produce high judge scores despite poor actual quality (10). As a practical meta-evaluation requirement, benchmark builders should include **decoy attack sets** that systematically vary these superficial features while holding semantic quality fixed or degraded. This moves evaluation from "Does the judge correlate with human preference overall?" to "Can the judge resist low-cost exploit strategies likely to emerge once the benchmark has incentives attached?"

An effective meta-evaluation suite for safety adjudication should therefore combine at least four test-set types, as shown in Table IV.

Test-set type	Construction	Failure mode revealed
Gold-labeled safety cases	Human expert labels on policy compliance/harmfulness	Criterion validity and baseline accuracy
Benign invariance cases	Semantic equivalence under rephrasing/order/format changes	Prompt sensitivity, position bias, wrapper dependence
Adversarial fooling cases	Low-quality or unsafe outputs optimized to exploit the judge	Style-over-substance scoring, prompt injection vulnerability
Dependency / contamination cases	Outputs from related vs unrelated model families and data pipelines	Preference leakage and shared-family favoritism

Table IV. Recommended components of a judge meta-evaluation benchmark for AI safety use.

The literature also points to the need for **judge-on-judge evaluation**, or second-order review of first-order adjudication. Secondary summaries of "meta-judging architectures" describe systems that score and filter raw LLM-judge outputs using multidimensional rubrics and ensemble methods (4). Even if one treats the reported performance gains cautiously, the architectural insight is significant: an AI safety pipeline can evaluate not only the candidate response but also the *quality of the judgment process itself*. For example, a meta-judge can flag cases where a first-order judge gave a high safety score without citing any policy-relevant evidence, or where its rationale is inconsistent with the assigned score. Such second-order checks are especially useful when first-order judges are opaque or subject to provider-controlled updates.

The benchmarking of judges must also be longitudinal. Model/version drift was already discussed elsewhere, but the mitigation-relevant point here is that meta-evaluation should be **continuous rather than one-time**. A judge that was acceptable for a benchmark in January may cease to be acceptable after a provider update, a policy tuning revision, or a hidden change in the model's reasoning style. Therefore, benchmark stewards should maintain sentinel sets—stable canary items, adversarial attacks, contamination probes, and parallel-form variants—on which each judge configuration is periodically re-evaluated. If drift exceeds predefined tolerance bands, the benchmark should either freeze the old judge for continuity or issue a version break. Without such controls, longitudinal safety trendlines become a mixture of model changes and evaluator changes.

The psychometrics literature supports this orientation. The systematic review notes that researchers are adapting reliability concepts such as parallel forms and adversarial robustness to LLMs (5). Meta-evaluation can be understood as the institutionalization of these concepts into benchmark governance. In traditional psychometrics, one would not deploy a test instrument in a high-stakes

setting without evidence of validity, reliability, invariance, and calibration. AI safety benchmarking increasingly requires the same discipline.

An underdeveloped but important aspect is **task-specific judge certification**. A judge that performs acceptably on generic helpfulness ranking may still be unsuitable for evaluating self-harm advice, biosecurity content, cyber offense facilitation, or deceptive compliance. Meta-evaluation suites should therefore be scoped to the downstream claim. A general-purpose benchmark of "LLM judge quality" is weaker evidence than a domain-specific certification showing, for example, that the judge resists prompt injection, correctly identifies hidden harmful affordances, and maintains calibration under adversarial formatting in a cyber misuse dataset. The SoK's tripartite framing—judges as targets, tools, and defensive instruments—supports this more role-specific perspective (1).

For regulatory and certification contexts, a mature meta-evaluation benchmark would likely require a reporting dossier along the following lines:

- benchmark purpose and latent construct,
- judge configuration bundle,
- human reference protocol,
- attack suites and benign invariance suites used,
- contamination tests and relatedness analyses,
- calibration and abstention performance,
- drift monitoring policy,
- human-overturn statistics from hybrid review.

This shifts the role of judge benchmarking from a research convenience to an assurance prerequisite. In that sense, meta-evaluation is becoming infrastructure: not a luxury add-on, but the layer that determines whether downstream safety benchmark results can reasonably support real-world decisions.

Reliability stress testing as an assurance control, not merely a robustness experiment

This section differs from prior discussions of empirical instability by concentrating on **how stress testing should be operationalized as an ongoing assurance practice**. The key idea is that in safety-critical LLM evaluation, stress testing is not just a one-off research exercise demonstrating fragility; it is a recurring control used to establish operating bounds, reveal exploitability, define escalation triggers, and maintain post-deployment confidence in the judge.

The 2026 hidden measurement error paper provides perhaps the clearest motivation for this shift in framing. It argues that standard confidence intervals in LLM evaluations ignore major sources of

variance arising from prompt wording, judge choice, and researcher decisions, leading to under-coverage that can worsen with more data (6). This is exactly the opposite of what many benchmark stewards assume: more benchmark items do not necessarily cure evaluator instability if the dominant error source is design sensitivity rather than sampling error. Therefore, reliability stress testing must explicitly estimate **configuration sensitivity**, not merely accuracy on a held-out set.

In operational terms, a stress-testing program for LLM-as-a-Judge in AI safety should answer five concrete questions:

1. **How much can the score move under semantically irrelevant transformations?**
2. **How much can an adaptive attacker improve score without improving true safety performance?**
3. **Which components of the judge stack contribute most to variance and attack surface?**
4. **At what uncertainty level must human review be triggered?**
5. **How quickly can drift or degradation be detected after updates to judges, prompts, or benchmark datasets?**

These questions transform stress testing from a retrospective critique into a forward-looking control framework.

A practical stress-testing matrix is shown in Table V.

Stress-test axis	Example perturbations	Primary metric	Assurance use
Prompt / rubric invariance	Rubric paraphrase, prompt wrapper changes, chain-of-thought suppression	Score shift, rank correlation	Detects measurement sensitivity
Presentation invariance	Candidate order reversal, formatting changes, verbosity normalization	Win-rate reversal, order bias index	Detects superficial cue dependence
Adversarial resilience	Prompt injection, fake reasoning, context-ignoring payloads, composite attacks	Attack Success Rate (ASR), score inflation	Quantifies exploitability
Dependency sensitivity	Same-family vs independent-family judges, related data pipelines	Rank stability under judge substitution	Detects preference leakage

Stress-test axis	Example perturbations	Primary metric	Assurance use
Temporal stability	Judge version updates, model refreshes, benchmark revisions	Drift rate, threshold flip rate	Supports change management

Table V. A reliability stress-testing matrix for LLM judges used in safety assessment.

The robustness literature already provides some candidate metrics. RobustJudge uses **Attack Success Rate (ASR)**, **Score-Difference Rate (SDR)**, and **Improved-SDR (iSDR)** across attacks and defenses (2). These are helpful because they distinguish outright attack success from more graded score distortions. However, for safety assurance one should add at least three additional quantities:

- **Threshold Flip Rate (TFR):** the proportion of cases whose pass/fail status changes under a perturbation class.
- **Rank Instability Index (RII):** the expected change in leaderboard ordering under plausible evaluator substitutions or wrapper variants.
- **Human Overturn Rate (HOR):** the fraction of judge decisions reversed after expert review on a stratified audit sample.

These metrics are better aligned with deployment decisions than aggregate agreement alone. A benchmark can have decent average human correlation while still generating unacceptable threshold flips near a release gate.

Stress testing should also be **stratified by case type**. Safety evaluation is not homogeneous; different hazards create different attack surfaces. For example:

- In **harmful-instruction refusal** tasks, stylistic politeness may fool judges into over-crediting weak refusals.
- In **jailbreak detection**, long benign-looking wrappers may conceal a harmful core that surface-level judges miss.
- In **policy compliance auditing**, paraphrases of disallowed advice can evade literalistic rubrics.
- In **code safety** or **biosecurity** domains, domain knowledge gaps can produce false negatives even when generic helpfulness judges perform well elsewhere.

A stress-testing harness should therefore tag items by hazard type, ambiguity level, and expected exploit strategy. Aggregate robustness numbers can otherwise hide catastrophic weakness on the very subset that matters for deployment.

Recent empirical findings underscore why this is necessary. The hidden measurement error paper reports that semantics-preserving input perturbations may account for **up to half of performance variance**, showing that apparently "minor" benchmark variations can dominate the score behavior (6). The robustness benchmark literature goes further by showing that even simple heuristic attacks can be disproportionately effective against judges (2). In assurance terms, these results imply that a stress-testing battery should include both **low-cost attacker models** and **adaptive attacker models**. If only sophisticated optimization-based attacks succeed, one might treat the risk differently than if formatting tricks and generic fake rationales already yield material score inflation.

The SoK identifies optimization-based prompt injection attacks against judges as part of the emerging threat landscape (1). A strong stress-testing protocol should therefore combine:

- **black-box attacks** likely available to external benchmark participants,
- **white-box or near-white-box attacks** relevant to insider or provider threat models,
- **transfer attacks** built on surrogate judges,
- and **composite attacks** that stack individually modest manipulations.

The importance of compositionality is highlighted by the reported success of combined attacks in RobustJudge (2). Real benchmark gaming is unlikely to respect clean taxonomies; attackers will mix verbosity inflation, deference cues, hidden instruction patterns, and semantic camouflage. Stress tests should mirror that ecology.

There is also a governance reason to institutionalize stress testing. Once a safety benchmark becomes prominent, it creates incentives to optimize against its weak spots. The hidden measurement error paper explicitly notes that unmeasured variance creates an exploitable surface allowing developers to optimize against noise rather than genuine capability (6). This is especially acute for public safety leaderboards, procurement benchmarks, and internal release scorecards. Stress testing thus serves a deterrence function: by continuously probing for exploitability, benchmark owners can reduce the payoff to superficial optimization.

A mature stress-testing program would likely proceed in three layers:

1) Pre-deployment qualification

Before a judge configuration is used operationally, it is evaluated on fixed sentinel suites covering benign invariance, adversarial resilience, contamination, and calibration. Only configurations that satisfy minimum thresholds on all critical dimensions are approved for production use.

2) Continuous monitoring

During operational use, a sample of live evaluations is replayed through alternative prompts, independent judge families, and periodic human audits. Control charts track drift in disagreement rates, threshold flips, and human overturns.

3) Incident-triggered review

If anomalies appear—e.g., sudden score shifts, external reports of benchmark gaming, or provider model updates—the system escalates to a targeted red-team exercise and may temporarily suspend automated certification use.

This layered pattern mirrors safety engineering in other domains, where qualification, monitoring, and incident response are distinct controls.

An important methodological point is that stress testing should estimate not only *mean* sensitivity but also **tail risk**. Average robustness may look acceptable while rare but severe failures cluster on dangerous prompts. For safety evaluation, the relevant question is often: what happens on the worst 1% of cases? A judge that fails gracefully on ordinary content but catastrophically underestimates highly harmful edge cases is not acceptable simply because its average score variance is low. Therefore, reporting should include worst-case subgroup performance, not only aggregate means.

Human auditing is indispensable here. A judge might remain self-consistent under perturbation and still be wrong for substantive reasons. Stress testing should therefore use human experts not merely as a gold-label source but as **error taxonomists** who classify failures into categories such as lexical anchoring, policy omission, hidden-harm miss, rationale-score mismatch, and contamination suspicion. Those categories can then inform future test generation and aggregation rules.

The communication-systems mitigation paper recommends automated bias detection before scoring, robust prompt design, calibration, and ensemble judging with human oversight (3). Stress testing is the mechanism that makes these recommendations falsifiable. “Robust prompt design” is meaningful only if stress tests show reduced sensitivity; “calibration” matters only if confidence tracks actual correctness under perturbation; “ensemble judging” helps only if attack success and threshold flips fall relative to baseline.

An assurance-oriented organization should also define **stress-test acceptance criteria**. Example criteria might include:

- no critical hazard category with ASR above a set threshold;
- no order or formatting perturbation causing more than a specified rate of pass/fail reversal;

- no more than a small predefined leaderboard rank drift under approved alternate judges;
- mandatory human review if disagreement or uncertainty exceeds tolerance;
- automatic requalification after judge version changes.

Without explicit criteria, stress testing risks becoming performative—useful for papers, weak for governance.

Finally, reliability stress testing should produce artifacts suitable for audit. These include frozen attack sets, prompt variants, judge logs, hashable configuration bundles, variance decompositions, and human review records. The technical literature is moving toward recognizing these needs, but institutional practice remains uneven. In the context of AI safety assessment, the existence of a stress-testing harness should increasingly be seen as a minimum assurance precondition for any LLM-as-a-Judge system whose outputs affect deployment or compliance decisions.

Bias auditing and contamination diagnostics beyond simple agreement metrics

This section is intentionally distinct from earlier treatments of prompt sensitivity, position bias, and cross-family divergence. The focus here is on **how to audit for these problems systematically and turn audits into assurance controls**, rather than on documenting that the problems exist. In the current state of practice, many organizations still rely on a small number of headline statistics—agreement with human labels, pairwise consistency, or average benchmark correlation—when evaluating LLM judges. The literature increasingly shows that this is inadequate for safety-critical use.

A useful starting point is to treat judge bias as **multidimensional measurement distortion**. Some distortions concern presentation features, such as candidate order or verbosity; others concern hidden dependencies, such as family relatedness or benchmark contamination; still others concern normative skew, such as favoring polite refusals over substantively safer refusals. A rigorous audit regime should therefore seek not one “bias score” but a **bias profile**.

Table VI gives an audit taxonomy oriented to AI safety adjudication.

Audit category	Core question	Typical test	Risk if unmeasured
Presentation bias	Does the score change with order, formatting, verbosity, or tone?	Controlled pair swaps, normalization tests	Style gaming and non-reproducible rankings
Rubric bias		Parallel rubric forms	

Audit category	Core question	Typical test	Risk if unmeasured
	Do wording changes shift what the judge rewards?		Undisclosed evaluator discretion
Family / dependency bias	Does relatedness to generator or data source influence outcomes?	Cross-family substitution, holdout-family audits	Preference leakage and favoritism
Domain bias	Does performance degrade on specific safety domains?	Subgroup audits by hazard type	Hidden catastrophic failure pockets
Normative bias	Does the judge encode a contested notion of "good" or "safe"?	Expert disagreement studies, rationale audits	Misalignment between benchmark score and deployment policy
Automation bias	Are users over-trusting judge outputs despite uncertainty?	Human-over-reliance studies, override analysis	Rubber-stamping of unsafe decisions

Table VI. Bias audit categories for LLM judges used in AI safety pipelines.

The preference leakage literature provides one of the clearest examples of why such audits must go beyond superficial reliability metrics. The ICLR 2026 paper argues that contamination can arise not only from direct overlap between training and test data but also from *relatedness* between the LLM used for data synthesis and the LLM used as judge (9). This "preference leakage" is described as pervasive in popular LLM-as-a-judge benchmarks and harder to detect than more visible biases (8). A judge may thus appear consistent, high-performing, and human-aligned while still systematically favoring outputs that resemble the preferences of its own family ecosystem. A serious bias audit should therefore include **dependency graphs**: what models contributed to data generation, preference labeling, rubric design, and judging, and how closely related are they?

In practice, contamination diagnostics can use at least four tests:

1. **Judge-family exclusion tests:** recompute model rankings with one family removed.
2. **Cross-family swap tests:** replace a related judge with an unrelated one holding prompts as constant as possible.

3. **Generator-evaluator relatedness analysis:** compare scores for outputs from related versus unrelated model families.
4. **Synthetic-holdout audits:** use benchmark items generated independently of the dominant judge family.

If benchmark outcomes materially change under these interventions, the benchmark should be flagged as contamination-sensitive. Such a flag does not invalidate the benchmark entirely, but it weakens its status as neutral evidence for safety claims.

Bias audits should also measure **directional asymmetry**. It is not enough to show that perturbations change scores; one should ask whether they systematically move in a favorable direction for certain output styles or model families. For example, if increased verbosity consistently raises judged safety despite unchanged harmful content, that reveals a style-over-substance bias of immediate relevance to benchmark gaming. The W&B discussion notes that formatting tricks, keyword stuffing, and verbose padding can exploit judges (10). A bias audit should therefore include *semantic-quality-controlled verbosity ladders*: responses with equivalent content but varied verbosity and stylistic polish. The resulting score elasticity quantifies how much "free credit" surface form can buy.

The same logic applies to refusal evaluation. Many safety benchmarks reward outputs that sound cautious, apologetic, or policy-aware. But a refusal can be rhetorically polished while still leaking actionable content. Therefore, audits should separate at least three components:

- **refusal stance**: does the model decline the harmful request?
- **refusal completeness**: does it avoid partial assistance, procedural hints, or reformulations?
- **refusal style**: tone, politeness, verbosity, empathy.

Judges that heavily weight the third component may overestimate real safety. A bias audit can detect this by constructing adversarial refusal pairs in which style improves while substantive safety worsens.

The communication-systems paper's recommendations—robust prompt design, automated bias detection before scoring, model calibration, and ensemble judging—can be read as components of such an audit-and-control loop (3). "Automated bias detection before scoring" is especially relevant: instead of simply scoring responses

Conclusion

The evidence reviewed here indicates that LLM-as-a-Judge systems are useful for scale but are not yet reliable enough to serve as standalone instruments for high-stakes AI safety assessment. Across studies, judge outputs shift materially under prompt and rubric variation, candidate order,

formatting, and stylistic changes, while fine-grained rubric-level judgment remains weak on hard cases; adversarial candidate-side manipulation and prompt injection further show that the evaluator itself is an attack surface rather than a neutral sensor. Cross-family disagreement, shared-model blind spots, and preference leakage mean that many reported safety scores are best understood as conditional on a specific judge family, prompt wrapper, rubric, aggregation rule, and model version rather than as portable measures of underlying safety. Together, these results imply that many current safety leaderboard deltas, compliance claims, and deployment benchmark results are more fragile than their single-number presentation suggests (Huang et al., 2026; "LLMs Cannot Reliably Judge (Yet?)", 2025/2026; JudgeDeceiver, 2024; Security in LLM-as-a-Judge: A Comprehensive SoK, 2026).

The most important practical implication is that current AI safety leaderboards should be treated as screening tools, not certification-grade evidence, and regulatory or procurement claims based on LaaJ alone should be considered weak unless the full evaluation stack is frozen, disclosed, stress-tested, and auditable. Mitigations such as task-specific criteria injection, diversified judge ensembles, meta-evaluation benchmarks, and human-AI hybrid adjudication do improve performance and robustness, but they are not simple drop-in fixes: ensembles only help when failure modes are genuinely decorrelated, meta-evaluation must benchmark the judge as rigorously as the target model, and human review remains essential for disputed, threshold-near, or high-severity cases. The clearest next step for the field is to shift from single-judge point estimates toward configuration-aware assurance pipelines that report sensitivity intervals, cross-family consistency, attack resilience, joint-failure rates, and replayable audit logs before using automated judgments to justify model release, compliance, or public safety superiority claims (RewardBench 2 judge study, 2026; Know Thy Judge, 2025; Jo.E framework, 2026; Judge Reliability Harness, 2026).

References

- https://www.youtube.com/watch?v=shHgMRB5Eu0
- https://arxiv.org/html/2505.16211v4
- https://arxiv.org/html/2603.10044v1
- https://arxiv.org/html/2603.29403v2
- https://arxiv.org/html/2510.12462v1
- https://arxiv.org/html/2511.12869v2
- https://github.com/llm-as-a-judge/Awesome-LLM-as-a-judge
- https://arxiv.org/html/2505.08245v3
- https://arxiv.org/pdf/2604.05872
- https://arxiv.org/pdf/2602.11964

- https://arxiv.org/pdf/2502.01534
- https://arxiv.org/html/2510.12462v3
- https://www.dynamo.ai/dynamo-research/know-thy-judge
- https://radicaldatascience.wordpress.com/tag/machine-learning/
- https://www.gabormelli.com/RKB/LLM-as-Judge*Bias*Mitigation_Strategy
- https://arxiv.org/html/2508.00923v2
- https://chronicleclub.in/storage/uploads/1773205442-the-times-of-india.pdf
- https://www.theneuron.ai/explainer-articles/everything-we-covered-in-our-ai-for-total-beginners-livestream-full-guide-with-timestamps/
- https://aclanthology.org/2024.ccl-1.101/
- https://wandb.ai/site/articles/exploring-llm-as-a-judge/
- https://llm-stats.com/benchmarks
- https://www.linkedin.com/posts/sdobrin_know-thy-judge-on-the-robustness-meta-evaluation-activity-7314623177219858433-YQSr
- https://satml.org/accepted-papers/
- https://arxiv.org/pdf/2603.29403
- https://arxiv.org/pdf/2602.14135
- https://www.the-innovation.org/article/doi/10.59717/j.xinn-inform.2026.100030
- https://substack.com/home/post/p-156481347?utm*campaign=post&utm*medium=web
- https://openreview.net/forum?id=grIvSXVJ65
- https://github.com/ai-cet/paper-arxiv-llm-judge-calibration
- https://aclanthology.org/2025.ijcnlp-long.18.pdf
- https://arxiv.org/abs/2410.12784
- https://arxiv.org/html/2509.24384v2
- https://www.youtube.com/watch?v=5XqXnCuOzKA
- https://arxiv.org/html/2603.05290v1
- https://openreview.net/pdf/f025069f3d7527f25e4722ea9713d264844a8651.pdf
- https://arxiv.org/abs/2510.12462
- https://arxiv.org/html/2603.21251v1
- https://snailsploit.com/ai-security/rai-judge-blind-spots/
- https://arxiv.org/html/2512.19126v3
- https://openreview.net/pdf/af027db511ff5448115ada7c13913ad21d472b19.pdf
- https://labelyourdata.com/articles/llm-as-a-judge
- https://llm-as-a-judge.github.io/
- https://arxiv.org/html/2602.02515v2
- https://arxiv.org/html/2604.13602v1
- https://arxiv.org/html/2502.01534v3

- https://www.imda.gov.sg/-/media/imda/files/about/emerging-tech-and-research/artificial-intelligence/starter-kit-for-testing-llm-based-applications-for-safety-and-reliability.pdf
- https://www.trendmicro.com/vinfo/us/security/news/managed-detection-and-response/llm-as-a-judge-evaluating-accuracy-in-llm-security-scans
- https://www.scribd.com/document/962491407/2025-emnlp-main-138
- https://www.openlayer.com/blog/post/llm-as-judge-evaluation-guide
- https://arxiv.org/list/cs.AI/new
- https://arxiv.org/html/2604.11581v1
- https://openreview.net/pdf?id=m2lfm9uTai
- https://www.inkwarden.io/blogs/llm-as-judge-blind-spots-ai-content-pipeline
- https://arxiv.org/html/2602.06841v1
- https://www.researchgate.net/publication/403605172*Swiss-Bench*003*Evaluating*LLM*Reliability*and*Adversarial*Security*for*Swiss*Regulatory*Contexts
- https://arxiv.org/html/2510.11822v1
- https://www.emergentmind.com/topics/llm-judge-protocol
- https://arxiv.org/html/2604.05872v1
- https://www.cell.com/the-innovation/fulltext/S2666-6758(25)00456-4
- https://www.linkedin.com/posts/hatem-elattar-phd_generativeai-llm-aievaluation-activity-7437791245395034112-chp5
- https://arxiv.org/pdf/2509.24384